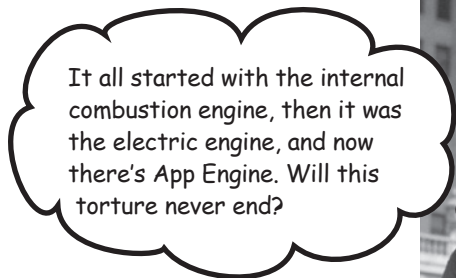


10 scaling your webapp

✦ *Getting real* ✦



The Web is a great place to host your app...until things get real.

Sooner or later, you'll hit the jackpot and your webapp will be *wildly successful*. When that happens, your webapp goes from a handful of hits a day to thousands, possibly ten of thousands, *or even more*. Will you be ready? Will your web server handle the **load**? How will you know? What will it **cost**? *Who will pay*? Can your data model scale to millions upon millions of data items without *slowing to a crawl*? Getting a webapp up and running is easy with Python and now, thanks to Google App Engine, **scaling** a Python webapp is achievable, too. So...flip the page and find out how.

There are whale sightings everywhere

The Head First Whale Watching Group (HFWWG) coordinates the live cetacean sightings for the entire country. To date, they’ve provided a PDF form on their website that members of the public can download, fill in, and mail to the HFWWG central office.

The form contains the essential data needed to record the sighting:

Head First Whale Watching Group
Casual Sighting Form


HFWWG.ORG

Name:

Email:

Date:

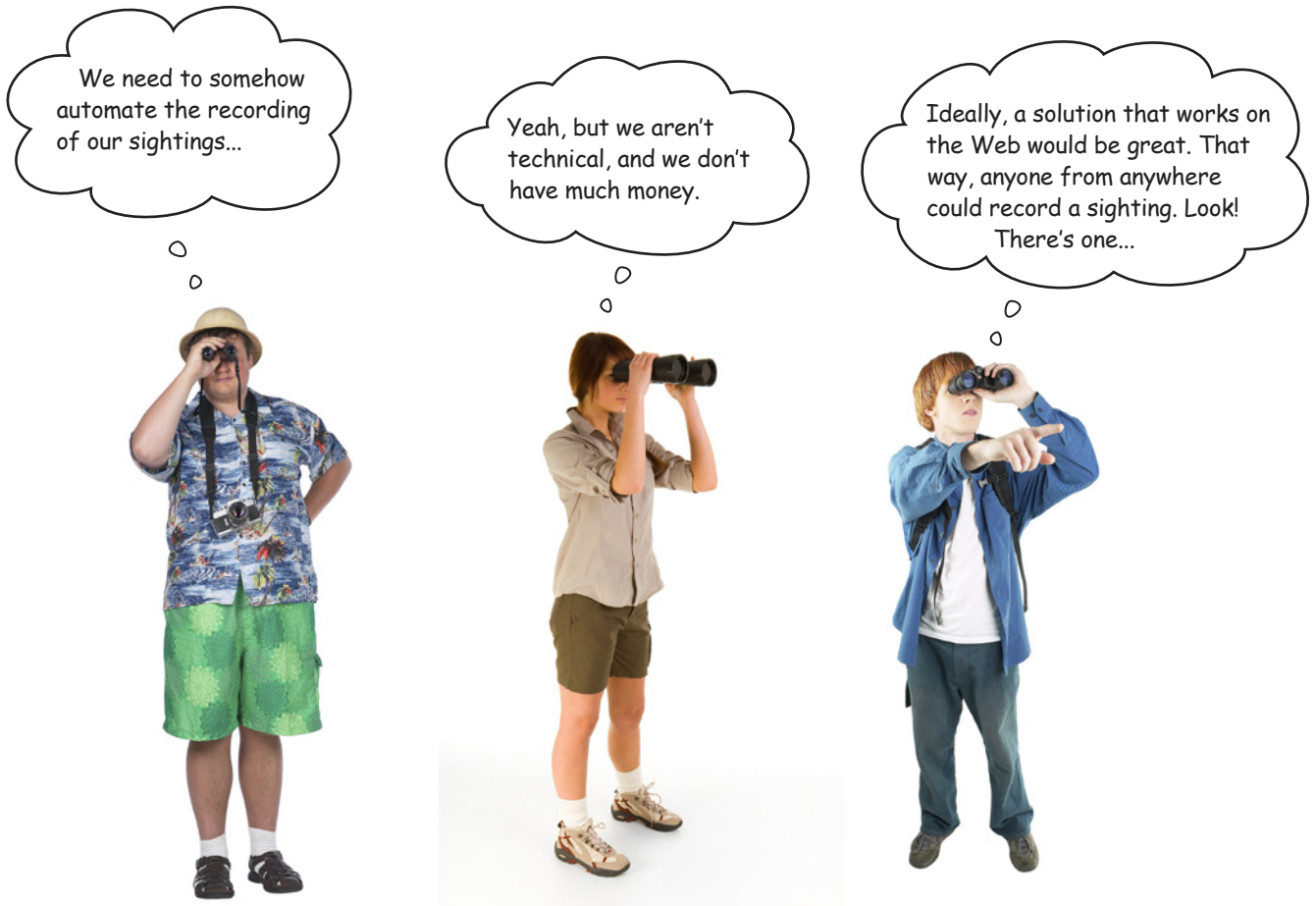
Time:

Location:

Fin Type:	Falcate	Triangular	Rounded	
Whale Type:	Humpback	Orca	Blue	Killer
	Beluga	Fin	Gray	Sperm
Blow Type:	Tall	Bushy	Dense	
Wave Type:	Flat	Small	Moderate	
	Large	Breaking	High	

After a busy sightings weekend, the central office is swamped with completed forms for **thousands** of sightings...which is a *data-entry nightmare* as all those forms can take an age to process manually. There’s nothing worse than being stuck in front of your computer entering data when all you want to do is be out on the water looking for humpbacks...

The HFWWG needs to automate



Suggesting to the HFWWG that they invest in an expensive web hosting solution isn't going to make you any friends. It's way too expensive to buy the capacity they'll need for the busy weekends and a total waste of capacity when sightings are infrequent.

Suggesting that they invest in a large, state-of-the-art web server that can be hosted in the central office is also a nonstarter: there's no one to look after a setup like that, and the broadband link required to handle the anticipated traffic would blow their budget right out of the water.

Is there another option?

Build your webapp with Google App Engine

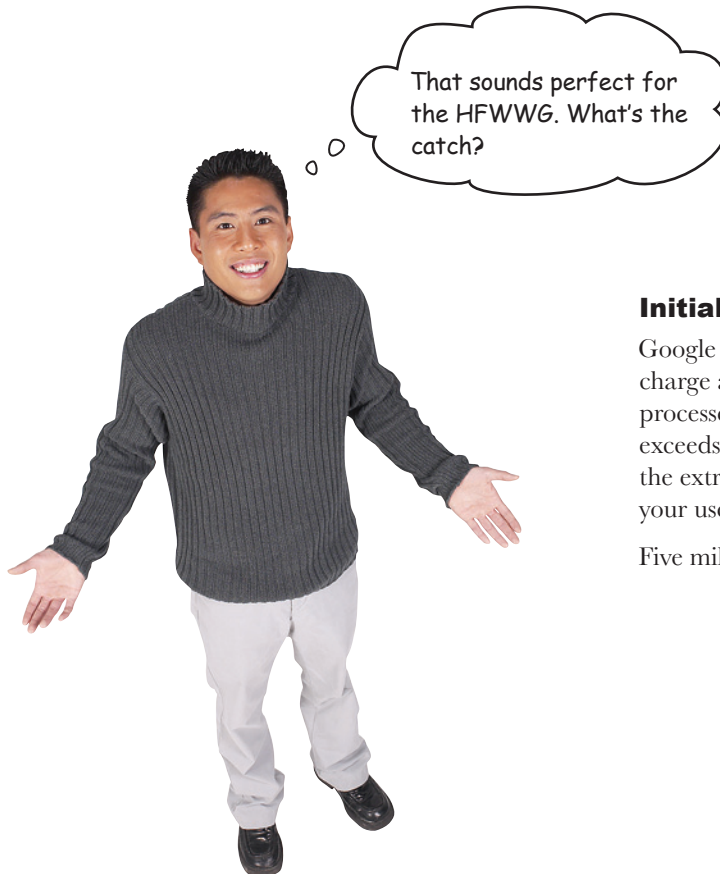
Google App Engine (GAE) is a set of technologies that lets you host your webapp on Google's cloud computing infrastructure.

GAE constantly monitors your running webapp and, based on your webapp's current activity, adjusts the resources needed to serve up your webapp's pages. When things are busy, GAE increases the resources available to your webapp, and when things are quiet, GAE reduces the resources until such time as extra activity warrants increasing them again.

On top of this, GAE provides access to Google's *BigTable* technology: a set of database technologies that make storing your webapp's data a breeze. Google also backs up your webapp's data on a regular basis, replicates your webapp over multiple, geographically dispersed web servers, and keeps App Engine running smoothly 24/7.

And the *best part*? GAE can be programmed with Python.

And the *even better part*? You can start running your webapp on GAE *for free*.



Initially, there isn't one.

Google provides this webapp hosting service at no charge and will continue to do so until your webapp processes five million page views per month. Once it exceeds this threshold, you'll need to pay Google for the extra capacity used. If you never reach the limit, your use of GAE is not charged.

Five million page views? That's a lot of sightings...

Download and install App Engine

When your webapp is ready for deployment, you'll upload it to the Google cloud and run it from there. However, during development, you can run a test version of your webapp locally on your computer. All you need is a copy of the GAE SDK, which is available from here:



<http://code.google.com/appengine/>

Download the GAE Python SDK for your operating system. Windows, Mac OS X, and Linux are all supported, and installation is straightforward.

After installation, Windows and Mac OS X users will find a nice, graphical front end added to their system.

On Linux, a new folder called "google_appengine" is created after a successful install.

GAE uses Python 2.5

The version of Python built into GAE is a modified version of the Python 2.5 release. As when you worked with Python for Android, the fact that you aren't running Python 3 isn't such a big deal with GAE, although you do need to ensure Python 2.5 is installed on your computer. Open up a terminal window and type:

```
python2.5 -V
```

If this command gives an error, pop on over to the Python website and grab the 2.5 release for your operating system.

there are no
Dumb Questions

Q: Aren't things going backward here? First, there was Python 3, then it was Python 2.6 for Android, and now we are dropping down to 2.5 for App Engine? What gives?

A: That's a great question. It's important to remember to always code to the restrictions placed on you. You might think that it sucks that GAE runs on Python 2.5, but you shouldn't. Think of it as just another restriction placed on the code you write—that is, it must target Release 2.5 of Python. As with the Android code you created in the previous chapters, the GAE code you are about to write is not all that different than the Python code for 3. In fact, you will be hard pressed to spot the difference.

Make sure App Engine is working

The environment supported by GAE within the Google cloud supports standard CGI or Python's WSGI. To build a GAE-compatible webapp, you need three things: a **folder** to hold your webapp's files, some **code** to execute, and a **configuration file**.

To test your setup, create a folder called `mygaetest`. Within the folder, create a small CGI you can use to test GAE. Call this CGI `sayhello.py`. Use this code:

```
print('Content-type: text/plain\n')

print('Hello from Head First Python on GAE!')
```

It doesn't get much easier than this...a plain-text message is displayed within your browser whenever this CGI runs.

The configuration file *must* be called `app.yaml` and it, too, must be in your webapp's folder. This file tells the Google cloud a little bit about your webapp's runtime environment. Here's a basic configuration file:

The "application" line identifies your webapp and is the same name as your folder.

"runtime" tells GAE that your webapp is written in and will run on Python.

Think of the "handlers" section of the configuration file as a top-level webapp routing mechanism.

```
application: mygaetest
version: 1
runtime: python
api_version: 1

handlers:
- url: /*
  script: sayhello.py
```

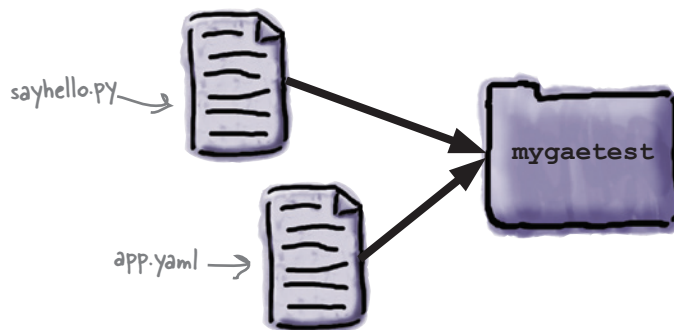
The "version" line identifies the current version of your webapp (and usually starts at 1).

The "api_version" indicates the release of GAE you are targeting.

This entry tells GAE to route all requests to your webapp to your "sayhello.py" program.



Go ahead and create the folder called `mygaetest` and the two files shown here.

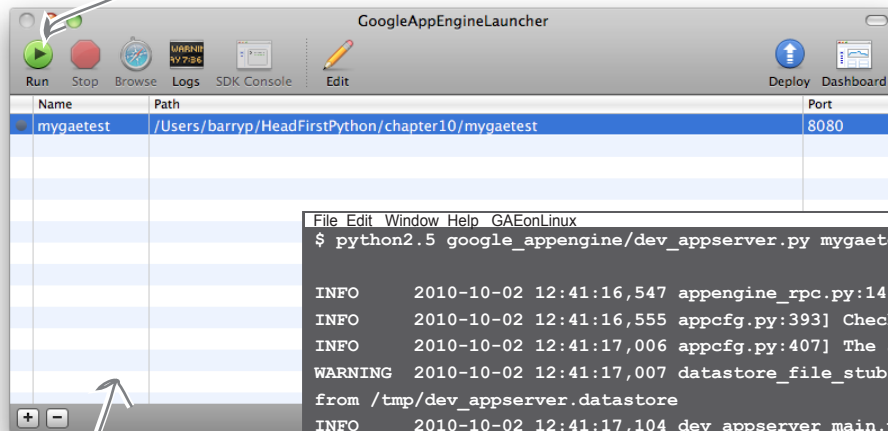




Test Drive

The GAE SDK includes a test web server, so let's use it to take your test GAE webapp for a spin. If you are running on Windows or Mac OS X, fire up the Google App Engine Launcher front end. This tool makes it easy to start, stop, and monitor your webapp. On Linux, you'll need to invoke a command to kick things off. If you are using the GAE Launcher, choose File -> Add Existing Application from the menu system to browse and select your webapp's folder. Also: be sure to edit the Launcher's Preferences to select Python 2.5 as your preferred Python Path.

Click this button to start your webapp.



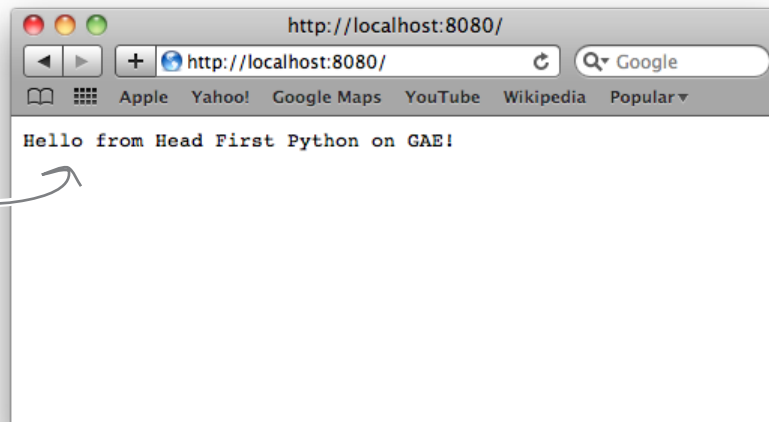
There is no graphical front end for Linux, so start your GAE webapp from the command line.

```
File Edit Window Help_GAEonLinux
$ python2.5 google_appengine/dev_appserver.py mygaetest/

INFO     2010-10-02 12:41:16,547 appengine_rpc.py:149] Server: appengine.google.com
INFO     2010-10-02 12:41:16,555 appcfg.py:393] Checking for updates to the SDK.
INFO     2010-10-02 12:41:17,006 appcfg.py:407] The SDK is up to date.
WARNING  2010-10-02 12:41:17,007 datastore_file_stub.py:657] Could not read datastore data
from /tmp/dev_appserver.datastore
INFO     2010-10-02 12:41:17,104 dev_appserver_main.py:431] Running application mygaetest
on port 8080: http://localhost:8080
```

This is how the GAE Launcher looks on Mac OS X...it looks similar on Windows.

With your webapp running and waiting on port 8080, open your favorite web browser and surf on over to the <http://localhost:8080/> web address.



And there it is... the message from your test webapp!



I don't believe it. This is actually more work than plain old CGI...and you're claiming this is better?!?

Yes, it is more work. But that's about to change.

For now, this is more work than you're used to, but remember that this is just a quick test to make sure your GAE test environment is up and running (and it is). When you start to work with some of GAE's web development features, you'll initially see that there's a lot more going on *behind the scenes* than meets the eye.

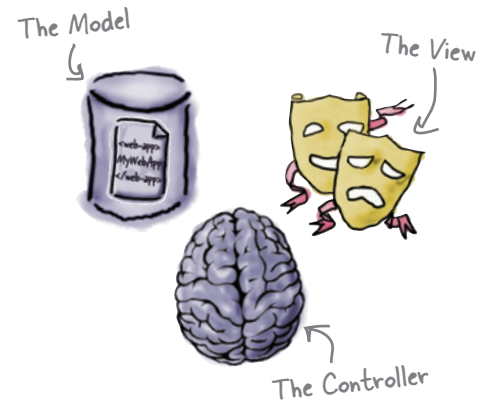
App Engine uses the MVC pattern

Google has built GAE to conform to the familiar Model-View-Controller (MVC) pattern.

Like your webapp from the previous chapter, the model component of a GAE-enabled webapp uses a back-end data storage facility that's known as the **datastore**. This is based on Google's *BigTable* technology, which provides a "NoSQL" API to your data, as well as a SQL-like API using Google's Query Language (GQL).

GAE's views use templates, but unlike the simple string templates from the previous chapter, GAE uses the **templating system** from the Django Project, which is one of Python's leading web framework technologies. In addition to templates, GAE includes Django's forms-building technology.

And, of course, any controller **code** is written in Python and can use the CGI or WSGI standards. Unfortunately, you can't use your *yate* module with GAE, because it is a Python 3 library (and would need to be extensively rewritten to support Python 2). Not to worry: the facilities provided by GAE "out of the box" are more than enough to build *great* webapps.



So...like any other webapp that I build, with App Engine I define a **model** for my data, create some **templates** for my view, and then control it all with **code**, right?

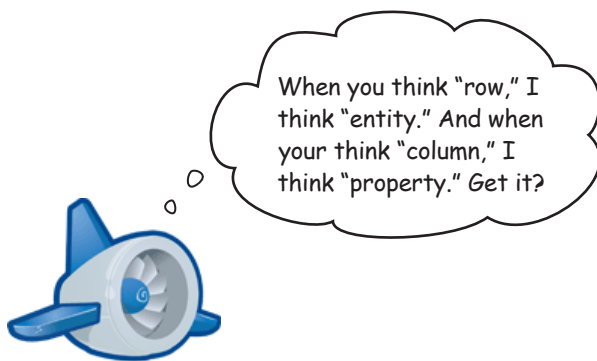
Yes, it's the same process as any other webapp.

Google has worked hard to ensure that the move to App Engine is as painless as possible. If you understand MVC (as you now do), you are well on your way to creating with GAE. It's just a matter of working out how GAE implements each of the MVC components.

Model your data with App Engine

App Engine refers to data items stored within its datastore as *properties*, which are defined within your model code.

Think of properties as a way to define the name and types of data within your database schema: each property is like the column type associated piece of data stored in a row, which App Engine refers to as an *entity*.



As with traditional SQL-based databases, your GAE datastore properties are of a specific, predeclared type. There are *lots* to choose from, for instance:

- **db.StringProperty**: a string of up to 500 characters
- **db.Blob**: a byte string (binary data)
- **db.DateProperty**: a date
- **db.TimeProperty**: a time,
- **db.IntegerProperty**: a 64-bit integer
- **db.UserProperty**: a Google account

For the full list of property types supported, pop on over to <http://code.google.com/appengine/docs/python/datastore/typesandpropertyclasses.html> and take a look.

Here's some sample data from the previous chapter.

Store this data as "db.StringProperty".

This data is stored as a "db.IntegerProperty".

This data is stored as a "db.DateProperty".

id	name	dob
1	James Lee	2002-03-14
2	Sarah Sweeney	2002-06-17
3	Vera Vi	2002-12-25
4	Julie Jones	2002-08-17
5	Sally Sanchez	2002-11-24
6	Mikey McManus	2002-02-24

Pool Puzzle



Your **job** is to take the properties from the pool and place them in the correct place in the class code, which is in a file called `hfwwgDB.py`. Your **goal** is to assign the correct property type to each of the attributes within your Sighting class.

Import the "db" module from the GAE extensions.

`from google.appengine.ext import db`

`class Sighting(db.Model):`

Create a class called "Sighting" that inherits from the GAE "db.Model" class.

Each property is assigned to a name.

```

name = .....
email = .....
date = .....
time = .....
location = .....
fin_type = .....
whale_type = .....
blow_type = .....
wave_type = .....

```

Head First Whale Watching Group
Casual Sighting Form



Name:

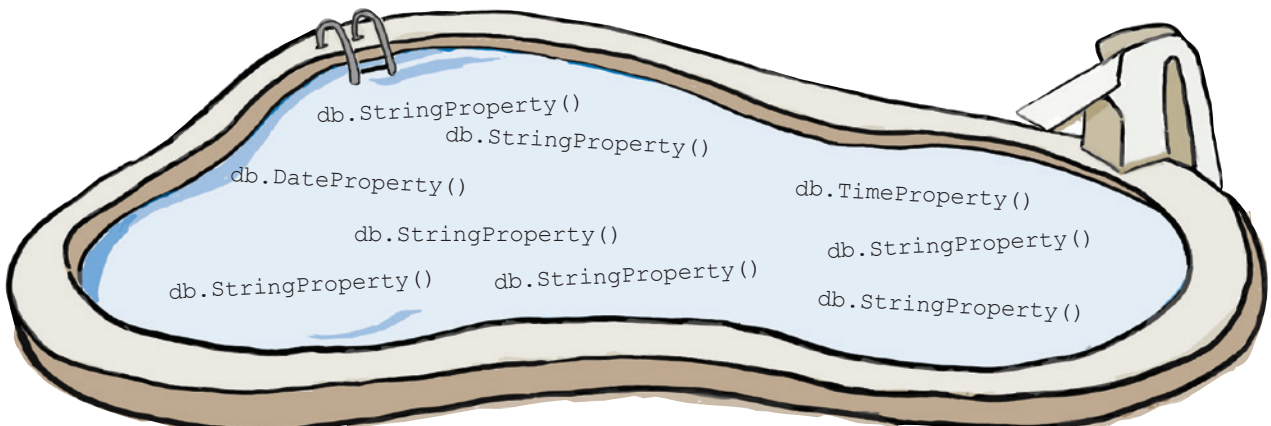
Email:

Date:

Time:

Location:

Fin Type:	Falcate	Triangular	Rounded
Whale Type:	Humpback	Orca	Blue
	Beluga	Fin	Gray
			Sperm
Blow Type:	Tall	Bushy	Dense
Wave Type:	Flat	Small	Moderate
	Large	Breaking	High



Pool Puzzle Solution



Your **job** was to take the properties from the pool and place them in the correct place in the class code, which is in a file called `hfwwgDB.py`. Your **goal** was to assign the correct property type to each of the attributes within your `Sighting` class.

Head First Whale Watching Group
Casual Sighting Form

Name:

Email:

Date:

Time:

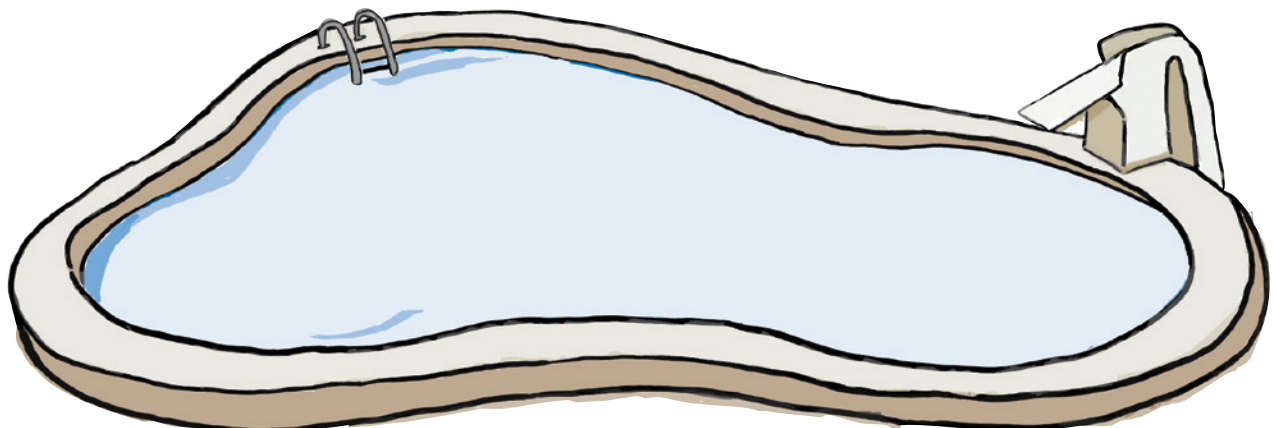
Location:

Fin Type:	Falcate	Triangular	Rounded	
Whale Type:	Humpback	Orca	Blue	Killer
	Beluga	Fin	Gray	Spinn
Blow Type:	Tail	Bushy	Dense	
Wave Type:	Flat	Small	Moderate	
	Large	Breaking	High	

```
from google.appengine.ext import db
```

```
class Sighting(db.Model):  
    name = db.StringProperty()  
    email = db.StringProperty()  
    date = db.DateProperty()  
    time = db.TimeProperty()  
    location = db.StringProperty()  
    fin_type = db.StringProperty()  
    whale_type = db.StringProperty()  
    blow_type = db.StringProperty()  
    wave_type = db.StringProperty()
```

Everything is a
"StringProperty",
except the "date" and
"time" fields.



What good is a model without a view?

GAE not only lets you *define* the schema for your data, but it also *creates* the entities in the datastore. The first time you go to **put** your data in the datastore, GAE springs to life and makes room for your data. There's no extra work required by you, other than defining your model in code. It's useful to think of GAE as executing something similar to a SQL `CREATE` command *on the fly* and *as needed*. But how do you get data into the GAE datastore?

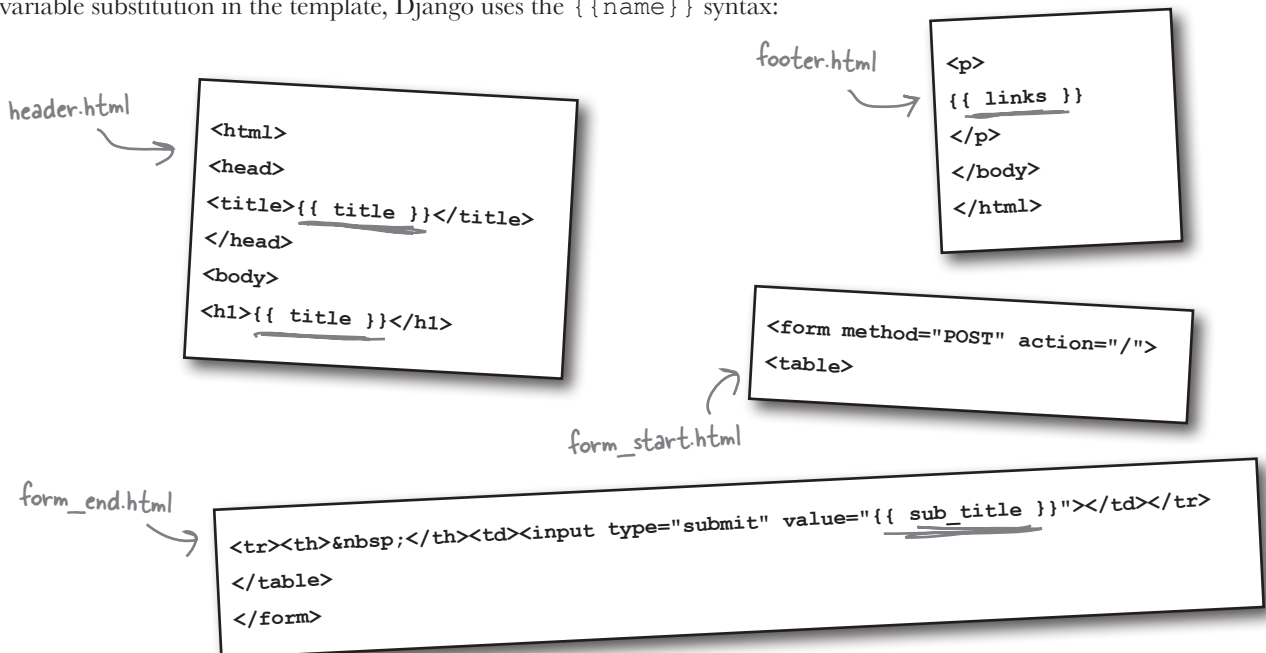


The short answer is that *you put it there*, but you first need to get some data from your webapp's user...and to do that, you need a *view*. And views are easy when you use *templates*.

App Engine templates in an instant

Recall that the templating technology built into GAE is based on technology from the Django Project. Django's templating system is more sophisticated than the simple string-based templates used in the previous chapter. Like your templates, Django's templates can substitute data into HTML, but they can also execute *conditional* and *looping* code.

Here are four templates you'll need for your HTWWG webapp. Two of them should be familiar to you: they are adaptations of those used in the previous chapter. The other two are new. Go ahead and grab them from this book's support website. As you can see, rather than using the `$name` syntax for variable substitution in the template, Django uses the `{{ name }}` syntax:



Use templates in App Engine

To use a template, import the `template` module from `google.appengine.ext.webapp` and call the `template.render()` function. It is useful to assign the output from `template.render()` to a variable, which is called `html` in this code snippet:

```
from google.appengine.ext.webapp import template
html = template.render('templates/header.html', {'title': 'Report a Possible Sighting'})
```

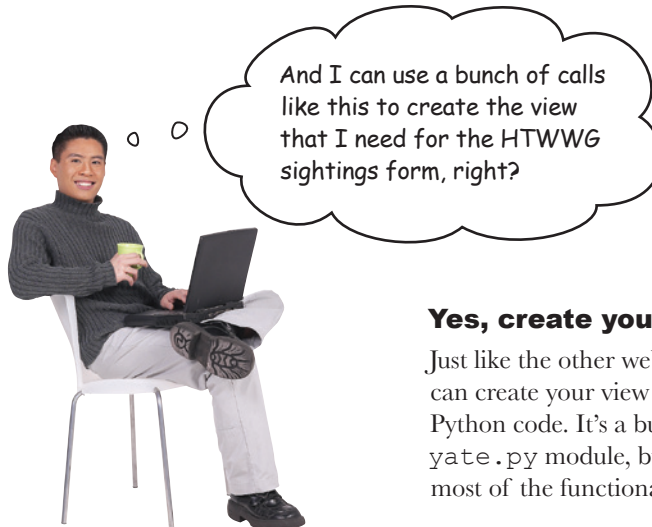
Call "template.render()"...

...supplying the template name...

As usual, start with your import.

...as well as a dictionary that maps values to the named template variable.

This is similar to the mechanism your `yate.py` module uses to parameterize the data displayed within your HTML pages.



Yes, create your view with templates.

Just like the other webapps that you've built, you can create your view in much the same way using Python code. It's a bummer that you can't use your `yate.py` module, but Django's templates provide most of the functionality you need here.

there are no
Dumb Questions

Q: Should I create one big template for my entire web page?

A: You could, if you want. However, if you build up your view from snippets of HTML in templates, you open up the possibility of reusing those HTML snippets in lots of places. For instance, to maintain a consistent look and feel, you can use the same header and footer template on all of your web pages, assuming of course that your header and footer aren't already embedded in an entire web page (which can't be reused).

Sharpen your pencil



- 1 Let's write the rest of the code needed to create a view that displays a data entry form for your HFWWG webapp.

In addition to your web page header code (which already exists and is provided for you), you need to write code that starts a new form, displays the form fields, terminates the form with a submit button, and then finishes off the web page. Make use of the templates you've been given and (here's the rub) do it all in *no more than four additional lines of code*.

This code goes into a new program called "hfwwg.py".

```
from google.appengine.ext.webapp import template
```

```
html = template.render('templates/header.html', {'title': 'Report a Possible Sighting'})
```

```
html = html +
```

Extend the contents of "html" with the rest of the HTML you need.

Remember: no more than 4 lines of code!

- 2 Now that you have attempted to write the code required in *no more than four lines of code*, what problem(s) have you encountered. In the space below, note down any issue(s) you are having.

```
.....
.....
.....
.....
```



Sharpen your pencil Solution

- 1 You were to write the rest of the code needed to create a view that displays a data entry form for your HFWWG webapp.

In addition to your webpage header code (which already exists and is provided for you), you were to write code with starts a new form, displays the form fields, terminates the form with a submit button, then finishes off the webpage. You were to make use of the templates you've been given and (here's the rub) you had to do it all in *no more than four more lines of code*.

```
from google.appengine.ext.webapp import template
```

```
html = template.render('templates/header.html', {'title': 'Report a Possible Sighting'})
```

The "render()" function always expects two arguments. If you don't need the second one, be sure to pass an empty dictionary.

```
html = html + template.render('templates/form_start.html', {})
```

This is an issue, isn't it?

We need to generate the FORM fields in here...but how?!?

```
html = html + template.render('templates/form_end.html', {'sub_title': 'Submit Sighting'})
```

```
html = html + template.render('templates/footer.html', {'links': ''})
```

- 2 Having attempted to write the code required in *no more than four lines of code*, you were to make a note of any issue(s) you encountered.

This is IMPOSSIBLE to do in just four lines of code, because there's no way to generate the FORM fields that I need. I can't even use the "do_form()" function from "yate.py", because that code is not compatible with Python 2.5... this just sucks!

You may have written something like this...assuming, of course, you haven't thrown your copy of this book out the nearest window in frustration. ☺



Wouldn't it be dreamy if I could avoid hand-coding a `<FORM>` and generate the HTML markup I need from an existing data model? But I know it's just a fantasy...

Django's form validation framework

Templates aren't the only things that App Engine "borrows" from Django. It also uses its form-generating technology known as the *Form Validation Framework*. Given a data model, GAE can use the framework to generate the HTML needed to display the form's fields within a HTML table. Here's an example GAE model that records a person's essential birth details:

This code is in a file called "birthDB.py".

```
from google.appengine.ext import db

class BirthDetails(db.Model):
    name = db.StringProperty()
    date_of_birth = db.DateProperty()
    time_of_birth = db.TimeProperty()
```

This model is used with Django's framework to generate the HTML markup needed to render the data-entry form. All you need to do is inherit from a GAE-included class called `djangoforms.ModelForm`:

```
from google.appengine.ext.webapp import template
from google.appengine.ext.db import djangoforms
import birthDB

class BirthDetailsForm(djangoforms.ModelForm):
    class Meta:
        model = birthDB.BirthDetails
        ...

html = template.render('templates/header.html', {'title': 'Provide your birth details'})
html = html + template.render('templates/form_start.html', {})
html = html + str(BirthDetailsForm(auto_id=False))
html = html + template.render('templates/form_end.html', {'sub_title': 'Submit Details'})
html = html + template.render('templates/footer.html', {'links': ''})
```

Import the forms library in addition to your GAE data model.

Create a new class by inheriting from the "djangoforms.ModelForm" class, and then link your new class to your data model.

Use your new class to generate your form.

There is some code missing from here...but don't worry: you'll get to it in just a moment. For now, just concentrate on understanding the links between the model, the view code, and the Django form validation framework.

Check your form

The framework generates the HTML you need and produces the following output within your browser.

It's not the prettiest web page ever made, but it works.

Provide your birth details

Name:

Date of birth:

Time of birth:

Use the View Source menu option within your web browser to inspect the HTML markup generated.

Source of: http://localhost:8081/

```
<html>
<head>
<title>Provide your birth details</title>
</head>
<body>
<h1>Provide your birth details</h1>
<form method="POST" action="/">
<table>
<tr><th>Name:</th><td><input type="text" name="name" /></td></tr>
<tr><th>Date of birth:</th><td><input type="text" name="date_of_birth" /></td></tr>
<tr><th>Time of birth:</th><td><input type="text" name="time_of_birth" /></td></tr>
<tr><th><td><input type="submit" value="Submit Details"></td></tr>
</table>
</form>
<p></p>
</body>
</html>
```

By setting "auto_id" to "False" in your code, the form generator uses your model property names to identify your form's fields.

The Django framework is smart enough to create sensible labels for each of your input fields (based on the names used in your model).

It's time to tie things all together with your controller code.

Controlling your App Engine webapp

Like your other webapps, it makes sense to arrange your webapp **controller** code within a specific folder structure. Here's one suggestion:



Your top-level folder needs to be named to match the "application" line in your webapp's "app.yaml" file.



Put all of your webapp's controller code and configuration files in here.



If you have static content, put it in here (at the moment, this folder is empty).



Put your HTML templates in here.

As you've seen, any CGI can run on GAE, but to get the most out of Google's technology, you need to code to the WSGI standard. Here's some *boilerplate* code that every WSGI-compatible GAE webapp starts with:

Import App Engine's "webapp" class.

```
from google.appengine.ext import webapp
from google.appengine.ext.webapp.util import run_wsgi_app
```

Import a utility that runs your webapp.

```
class IndexPage(webapp.RequestHandler):
```

```
    def get(self):
        pass
```

This method runs when a GET web request is received by your webapp.

This class responds to a web request from your web browser.

Create an new "webapp" object for your application.

```
app = webapp.WSGIApplication([('/', '*', IndexPage)], debug=True)
```

This is not unlike switching on CGI tracking.

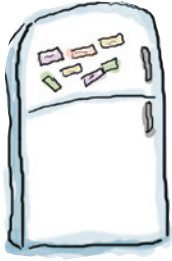
```
def main():
```

```
    run_wsgi_app(app)
```

Start your webapp.

```
if __name__ == '__main__':
    main()
```

Just use these two lines of code as-is.



App Engine Code Magnets

Let's put everything together. Your model code is already in your `hfwwgDB.py` file. All you need to do is move that file into your webapp's top-level folder. Copy your templates folder in there, too. Your webapp's controller code, in a file called `hfwwg.py`, also needs to exist in your top-level folder. The only problem is that some of the code's all over the floor. Rearrange the magnets to fix things.

```
from google.appengine.ext import webapp
from google.appengine.ext.webapp.util import run_wsgi_app
from google.appengine.ext import db
from google.appengine.ext.webapp import template
from google.appengine.ext.db import djangoforms
```

All of the imports
have survived...so
there's no need to
rearrange them.

Let's test how well
you've been paying
attention. There's no
guiding lines on the
fridge door.

What's missing
from in here?

```
html = template.render('templates/header.html', {'title': 'Report a Possible Sighting'})
html = html + template.render('templates/form_start.html', {})

html = html + template.render('templates/form_end.html', {'sub_title': 'Submit Sighting'})
html = html + template.render('templates/footer.html', {'links': ''})
```

```
app = webapp.WSGIApplication([('/',.*', SightingInputPage)], debug=True)
```

```
def main():
    run_wsgi_app(app)
```

```
if __name__ == '__main__':
    main()
```

There's only one small
change from the boilerplate
code in that "IndexPage" is
not being linked to.

```
class SightingForm(djangoforms.ModelForm):
```

```
html = html + str(SightingForm())
```

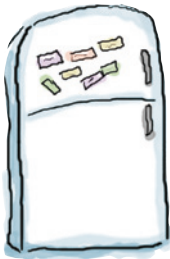
```
self.response.out.write(html)
```

```
class SightingInputPage(webapp.RequestHandler):
    def get(self):
```

```
model = hfwwgDB.Sighting
```

```
import hfwwgDB
```

```
class Meta:
```



App Engine Code Magnets Solution

Let's put everything together. Your model code is already in your `hfwwgDB.py` file. You were to move that file into your webapp's top-level folder, as well as copy your templates folder in there, too. Your webapp's controller code, in a file called `hfwwg.py`, also needs to exist in your top-level folder. The only problem is that some of the code's all over the floor. You were to rearrange the magnets to fix things:

```
from google.appengine.ext import webapp
from google.appengine.ext.webapp.util import run_wsgi_app
from google.appengine.ext import db
from google.appengine.ext.webapp import template
from google.appengine.ext.db import djangoforms
```

Import your GAE data model code.

```
import hfwwgDB
```

```
class SightingForm(djangoforms.ModelForm):
```

```
    class Meta:
```

```
        model = hfwwgDB.Sighting
```

Use your model to create a sighting form that inherits from the "django.ModelForm" class.

```
class SightingInputPage(webapp.RequestHandler):
    def get(self):
```

The connected handler class is called "SightingInputPage" and it provides a method called "get" which responds to a GET web request.

```
        html = template.render('templates/header.html', {'title': 'Report a Possible Sighting'})
```

```
        html = html + template.render('templates/form_start.html', {})
```

```
        html = html + str(SightingForm())
```

Include the generated form in the HTML response.

```
        html = html + template.render('templates/form_end.html', {'sub_title': 'Submit Sighting'})
```

```
        html = html + template.render('templates/footer.html', {'links': ''})
```

```
        self.response.out.write(html)
```

Did you guess this correctly? You need to send a response back to the waiting web browser and this line of code does just that.

```
app = webapp.WSGIApplication([('/', '*', SightingInputPage)], debug=True)
```

```
def main():
```

```
    run_wsgi_app(app)
```

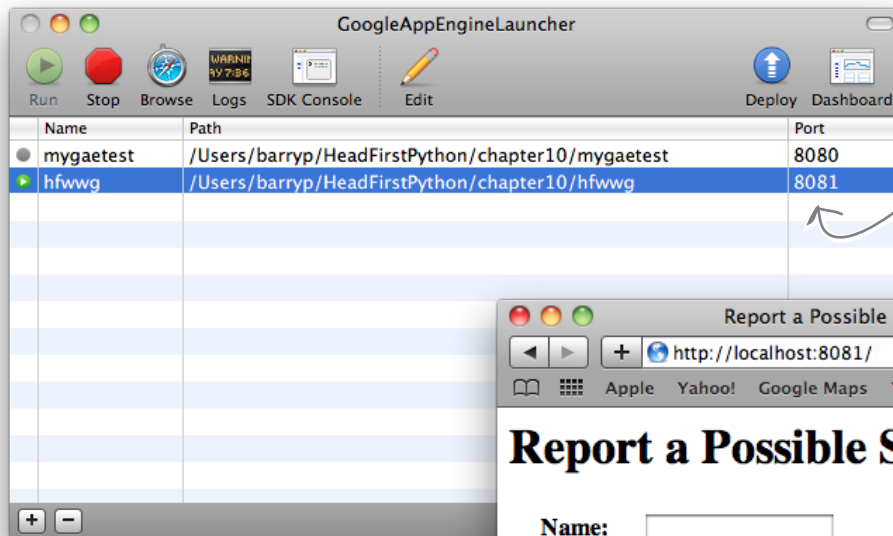
```
if __name__ == '__main__':
```

```
    main()
```



Test Drive

It's been a long time coming, but you are now ready to test the first version of your sightings form. If you haven't done so already, create an `app.yaml` file, too. Set the application line to `hfwwg` and the script line to `hfwwg.py`. One final step is to use the Add Existing Application menu option within the GAE Launcher to select your top-level folder as the location of your webapp.



The launcher adds your webapp into its list and assigns it the next available protocol port—in this case, 8081.

And here's your generated HTML form in all its glory.

Report a Possible Sighting

Name:

Email:

Date:

Time:

Location:

Fin type:

Whale type:

Blow type:

Wave type:

This is looking good. Let's get a quick opinion from the folks over at the HFWWG.



I know what you're thinking: "With a shirt like *that*, how can this guy possibly know anything about style?"... But let me just say that your form could do with a bit of, well...color, couldn't it? Any chance it could look nicer?

OK, we get it. Web design is not your thing.

Not to worry, you know all about code reuse, right? So, let's reuse *someone else's* **cascading style sheets** (CSS) to help with the "look" of your generated HTML form.

But who can you "borrow" from and not lose sleep feeling guilty over it?

As luck would have it, the authors of *Head First HTML with CSS & XHTML* created a bunch of stylesheets for their web pages and have made them available to you. Grab a slightly amended copy of some of their great stylesheets from this book's support website. When you unzip the archive, a folder called `static` appears: pop this entire folder into your webapp's top-level folder.

There's a file in `static` called `favicon.ico`. Move it into your top-level folder.

Improve the look of your form

To integrate the stylesheets into your webapp, add two `link` tags to your `header.html` template within your `templates` folder. Here's what the tags need to look like:

Add these two lines to the top of your "header.html" template.

```
<link type="text/css" rel="stylesheet" href="/static/hfwgw.css" />
<link type="text/css" rel="stylesheet" href="/static/styledform.css" />
```

GAE is smart enough to *optimize* the delivery of static content—that is, content that does *not* need to be generated by code. Your CSS files are static and are in your `static` folder. All you need to do is tell GAE about them to enable optimization. Do this by adding the following lines to the `handlers` section of your `app.yaml` file:

Provide the URL location for your static content.

```
- url: /static
  static_dir: static
```

Switch on the optimization.



Test Drive

With your stylesheets in place and your `app.yaml` file amended, ask your browser to **reload** your form.

Looking good. →

The screenshot shows a web browser window with the title 'Report a Possible Sighting'. The address bar shows 'http://localhost:8081/'. The browser's toolbar includes links to Apple, Yahoo!, Google Maps, and YouTube. The form itself has a title 'Report a Possible Sighting' in red. It contains several input fields with labels: 'Name:', 'Email:', 'Date:', 'Time:', 'Location:', 'Fin type:', 'Whale type:', 'Blow type:', and 'Wave type:'. At the bottom of the form is a button labeled 'Submit Sighting'.

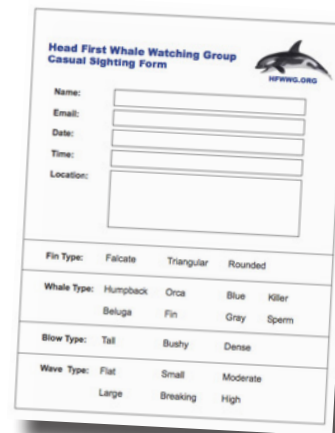
A little style goes
a long way...that's
looking great!



Restrict input by providing options

At the moment, your form accepts anything in the Fin, Whale, Blow, and Wave input areas. The paper form restricts the data that can be provided for each of these values. Your HTML form should, too.

Anything you can do to cut down on input errors is a good thing. As the youngest member of the group, I was “volunteered” to work on data clean-up duties...



Providing a list of choices restricts what users can input.

Instead of using HTML’s INPUT tag for all of your form fields, you can use the SELECT/OPTION tag pairing to *restrict* what’s accepted as valid data for any of the fields on your form. To do this, you’ll need more HTML markup. That’s the *bad news*.

The *good news* is that the form validation framework can generate the HTML markup you need for you. All you have to provide is the list of data items to use as an argument called `choices` when defining your property in your model code. You can also indicate when multiple lines of input are acceptable using the `multiline` argument to a property.

Apply these changes to your model code in the `hfwgDB.py` file.

This naming convention helps identify these lists as containing constant values.

```
_FINS = ['Falcate', 'Triangular', 'Rounded']
_WHALES = ['Humpback', 'Orca', 'Blue', 'Killer', 'Beluga', 'Fin', 'Gray', 'Sperm']
_BLOWS = ['Tall', 'Bushy', 'Dense']
_WAVES = ['Flat', 'Small', 'Moderate', 'Large', 'Breaking', 'High']
```

Define your lists of values near the top of your code.

```
...
location = db.StringProperty(multiline=True)
fin_type = db.StringProperty(choices=_FINS)
whale_type = db.StringProperty(choices=_WHALES)
blow_type = db.StringProperty(choices=_BLOWS)
wave_type = db.StringProperty(choices=_WAVES)
```

Switch on multiple-line input.

Use your lists of values when defining your properties.



Test Drive

With these changes applied to your model code, refresh your web browser once more.

Your form is not only looking good, but it's more functional, too.

The "location" field is now displayed over multiple lines.

Report a Possible Sighting

http://localhost:8081/ Google

Apple Yahoo! Google Maps YouTube

Report a Possible Sighting

Name: Eagle-eyed Jack Jones

Email: eej@hfwwg.org

Date: Jan 6, 2011

Time: 8:02am

Location: Just off the coast - really close to shore.

Fin type: Triangular

Whale type: Fin

Blow type: Bushy

Wave type: Breaking

Submit Sighting

Each of the "type" fields now have drop-down selection menus associated with them.

Your form now looks great! Go ahead and enter some test data, and then press the Submit Sighting button.

What happens?

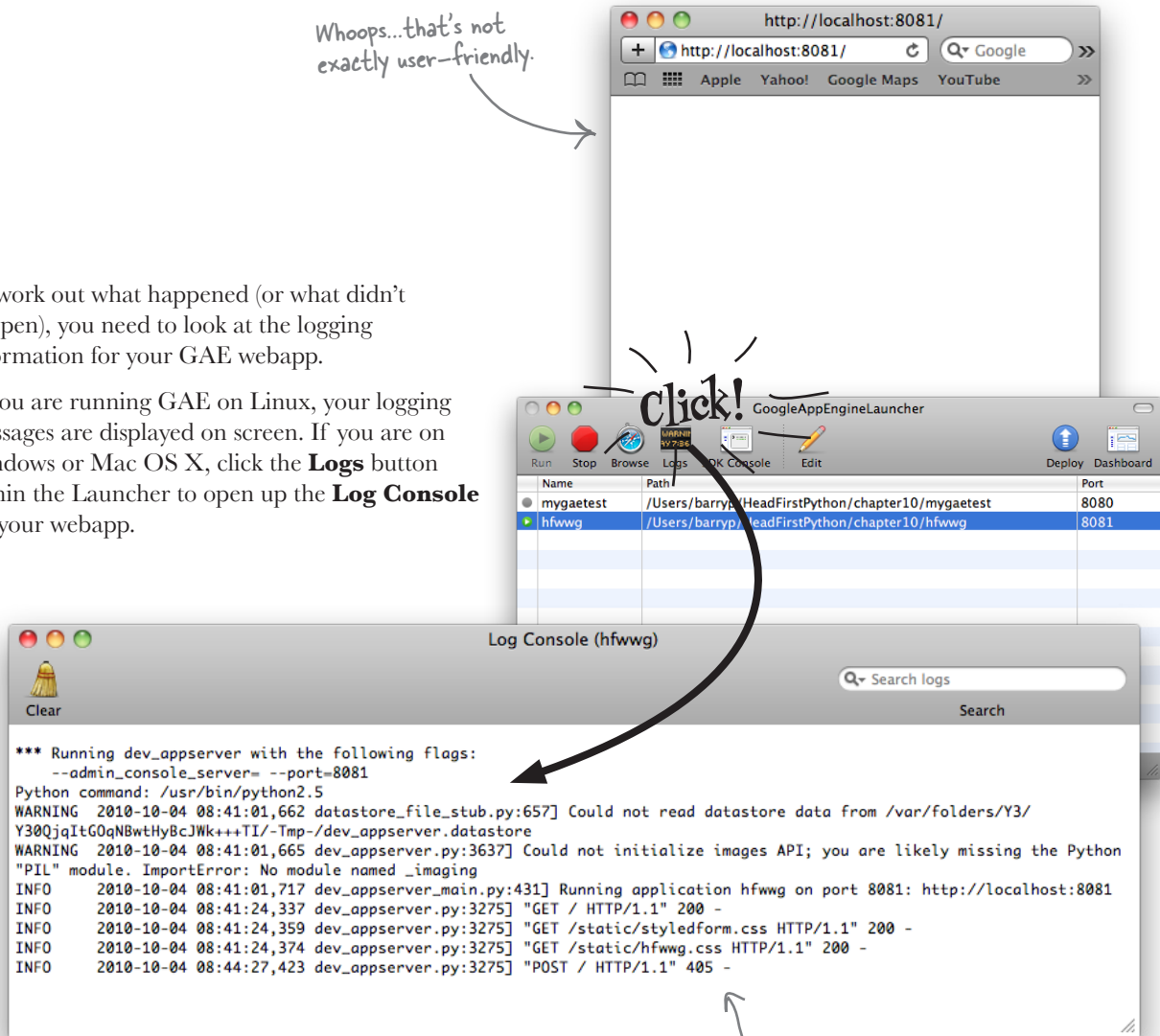
Meet the “blank screen of death”

Submitting your form’s data to the GAE web server produces a *blank screen*.

Whoops...that's not exactly user-friendly.

To work out what happened (or what didn't happen), you need to look at the logging information for your GAE webapp.

If you are running GAE on Linux, your logging messages are displayed on screen. If you are on Windows or Mac OS X, click the **Logs** button within the Launcher to open up the **Log Console** for your webapp.



Your request resulted in a 405 status code from the web server. According to the official HTTP RFC standards document, 405 stands for:

“Method Not Allowed. The method specified in the Request-Line is not allowed for the resource identified by the Request-URI. The response **MUST** include an Allow header containing a list of valid methods for the requested resource”.

Your last web request resulted in a 405.

Ummm...that's as clear as mud, isn't it?

Process the POST within your webapp

What the 405 status code actually tells you is that posted data arrived at your webapp intact, but that your webapp *does not* have any way of processing it. There's a method missing.

Take a quick look back at your code: the only method currently defined is called `get()`. This method is invoked whenever a GET web request arrives at your webapp and, as you know, it displays your sightings form.

In order to process posted data, you need to define *another* method. Specifically, you need to add a new method called `post()` to your `SightingInputPage` class.

Listen, bud, I'll happily process your web requests all day long...just as long as you give me the methods I need!



App Engine handles requests as well as responses

Your `get()` method produces your HTML form and returns a web response to the waiting web browser using the `self.response` object and by invoking the `out.write()` method on it.

In addition to helping you with your web responses, GAE also helps you process your web requests using the `self.request` object. Here are a few lines of code that displays all of the data posted to your web server:

Define a new method called "post".

Don't forget to use "self" with all your methods.

The "arguments()" method returns a list of the field names used on your form.

```
def post(self):
    for field in self.request.arguments():
        self.response.out.write(field)
        self.response.out.write(': ')
        self.response.out.write(self.request.get(field))
        self.response.out.write('<br />')
```

The "get()" method returns the value associated with the provided form field name.

So...if you know the *name* of your form field, you can access its value from within your webapp using the `self.request.get()` method.

But what do you do with the data once you have it?

Put your data in the datastore

Your data is sent to your webapp by GAE and you can use the `self.request.get()` method to access each input field value by name. Recall the `BirthDetails` model from earlier in this chapter:

This code is in a file called "birthDB.py".

```
from google.appengine.ext import db

class BirthDetails(db.Model):
    name = db.StringProperty()
    date_of_birth = db.DateProperty()
    time_of_birth = db.TimeProperty()
```

Assume that an HTML form has sent data to your webapp. The data is destined to be stored in the GAE datastore. Here's some code to do the heavy lifting:

```
def post(self):
    new_birth = birthDB.BirthDetails()

    new_birth.name = self.request.get('name')
    new_birth.date = self.request.get('date_of_birth')
    new_birth.time = self.request.get('time_of_birth')

    new_birth.put()

    html = template.render('templates/header.html', {'title': 'Thank you!'})
    html = html + "<p>Thank you for providing your birth details.</p>"
    html = html + template.render('templates/footer.html',
                                  {'links': 'Enter <a href="/">another birth</a>.'})

    self.response.out.write(html)
```

Create a new "BirthDetails" object to hold your data.

Get each of the form's data values and assign them to your new object's attributes.

Put (save) your data to the GAE datastore.

Generate a HTML response to say "thanks."

Send your response to the waiting web browser.

There's nothing to it: **create** a new object from your data model, **get** the data from your HTML form, **assign** it to the object's attributes, and then use the `put()` method to **save** your data in the datastore.



Exercise

Based on what you know about how to put your HTML form's data into the GAE datastore, create the code for the `post()` method that your webapp now needs. Some of the code has been done for you already. You are to provide the rest.

```
def post(self):
```

Put your code
here. 

```
html = template.render('templates/header.html',
                        {'title': 'Thank you!'})
html = html + "<p>Thank you for providing your sighting data.</p>"
html = html + template.render('templates/footer.html',
                              {'links': 'Enter <a href="/">another sighting</a>.'})

self.response.out.write(html)
```



Exercise Solution

Based on what you know about how to put your HTML form's data into the GAE datastore, you were to create the code for the `post()` method that your webapp now needs. Some of the code has been done for you already. You were to provide the rest.

```
def post(self):
    new_sighting = hfwwgDB.Sighting()

    .....

    new_sighting.name = self.request.get('name')
    new_sighting.email = self.request.get('email')
    new_sighting.date = self.request.get('date')
    new_sighting.time = self.request.get('time')
    new_sighting.location = self.request.get('location')
    new_sighting.fin_type = self.request.get('fin_type')
    new_sighting.whale_type = self.request.get('whale_type')
    new_sighting.blow_type = self.request.get('blow_type')
    new_sighting.wave_type = self.request.get('wave_type')

    .....

    new_sighting.put()
```

Create a new "Sighting" object.

For each of the data values received from the HTML form, assign them to the attributes of the newly created object.

Store your populated object in the GAE datastore.

```
html = template.render('templates/header.html',
                        {'title': 'Thank you!'})

html = html + "<p>Thank you for providing your sighting data.</p>"
html = html + template.render('templates/footer.html',
                              {'links': 'Enter <a href="/">another sighting</a>.'})

self.response.out.write(html)
```



Test Drive

Add your `post()` code to your webapp (within the `hfwwg.py` file) and press the **Back** button on your web browser. Click the Submit Sighting button once more and see what happens this time.

Here's your form with the data waiting to be submitted.

Report a Possible Sighting

Name:

Email:

Date:

Time:

But when you click the button, something bad has happened...your webapp has crashed.

```

Traceback (most recent call last):
  File "/Applications/GoogleAppEngineLauncher.app/Contents/Resources/GoogleAppEngine-de
    handler.post(*groups)
  File "/Users/barryp/HeadFirstPython/chapter10/hfwwg/hfwwg.py", line 27, in post
    new_sighting.date = self.request.get('date')
  File "/Applications/GoogleAppEngineLauncher.app/Contents/Resources/GoogleAppEngine-de
    value = self.validate(value)
  File "/Applications/GoogleAppEngineLauncher.app/Contents/Resources/GoogleAppEngine-de
    value = super(DateProperty, self).validate(value)
  File "/Applications/GoogleAppEngineLauncher.app/Contents/Resources/GoogleAppEngine-de
    (self.name, self.data_type.__name__)
BadValueError: Property date must be a date

```

It looks like you might have a problem with the format of your date property, doesn't it?

Phooey...that's disappointing, isn't it?

At the very least, you were expecting the data from the form to make it into the datastore...but something has stopped this from happening. What do you think is the problem?

Don't break the "robustness principle"

The **Robustness Principle** states: "*Be conservative in what you send; be liberal in what you accept.*" In other words, don't be too picky when *requesting* data of a certain type from your users, but when *providing* data, give 'em exactly what they need.

If you make it *too hard* for your users to enter data into your system, things will likely break. For instance, within your model code, consider how date and time are defined:

A date, and NOTHING
but a date will do.

```
...  
date = db.DateProperty()  
time = db.TimeProperty()  
...
```

You must provide a
valid value for time.
Anything else is simply
UNACCEPTABLE.

The trouble is, when it comes to dates and times, there are **lots** of ways to specify values.

Oh, la, la.. c'est temps
to toot mon flute! It's
14:00hr on 24/04/2011.



Get the low-down on
the hoedown: quarter
after six on 6/17/2011.



I say, old boy, tea is
at noon on the first
of each month.



Accept almost any date and time

If you are going to insist on asking your users to provide a properly formatted date and time, you'll need to do one of two things:

- Specify in detail the **format** in which you expect the data.
- **Convert** the entered data into a format with which you can work.

Both approaches have *problems*.

For example, if you are too picky in requesting a date in a particular format, you'll slow down your user and might end up picking a date format that is foreign to them, resulting in confusion.

If you try to convert *any* date or time entered into a common format that the datastore understands, you'll be biting off more than you can chew. As an example of the complexity that can occur, how do you know if your user entered a date in mm/dd/yyyy or dd/mm/yyyy format? (You don't.)

There is a third option

If your application doesn't require *exact* dates and times, *don't require them of your user*.

With your sightings webapp, the date and time can be *free-format fields* that accept any value (in any format). What's important is *the recording of the sighting*, not the exact date/time it occurred.

Of course, other webapps might not be as fast and loose with dates and times. When that's the case, you'll need to revert one of the options discussed earlier on this page and do the best you can.

Use "db.StringProperty()" for dates and times

If you relax the datatype restrictions on the date and time fields, not only do you make it easier on your user, but you also make it easier on you.

For the sightings webapp, the solution is to change the property type for date and time within the hfwgDB.py file from what they currently are to `db.StringProperty()`.

```
...
date = db.StringProperty()
time = db.StringProperty()
...
```

It's a small change, but it'll make all the difference.

Let's see what difference this change makes.



Test Drive

Change the types of date and time within `htwvgDB.py` to `db.StringProperty()`, being sure to save the file once you've made your edit. Click Back in your web browser and submit your sightings data once more.

OK, folks...
let's try
this again.

Success! It appears to
have worked this time.

Let's enter another
sighting, just to be
sure.

Report a Possible Sighting

http://localhost:8081/ Google

Apple Yahoo! Google Maps YouTube

Name: Eagle-eyed Jack Jones

Email: eej@hfwvg.org

Date: Jan 6, 2011

Time: 8:02am

Location: Just off the coast – really close to shore.

Fin type: Triangular

Whale type: Fin

Blow type: Bushy

Wave type: Breaking

Submit Sighting

Thank you!

http://localhost:8081/ Google

Apple Yahoo! Google Maps YouTube

Thank you!

Thank you for providing your sighting data.

Enter another sighting.

Report a Possible Sighting

http://localhost:8081/ Google

Apple Yahoo! Google Maps YouTube

Name: Mary Knighty

Email: mky@hfwvg.org

Date: Wed, Jan 12, 2011

Time: 9:43am

Location: Off Lighthouse Rock, looking north – east.

Fin type: Rounded

Whale type: Orca

Blow type: Dense

Wave type: Moderate

Submit Sighting

By relaxing the restrictions you placed on the types of data you'll accept, your webapp now appears to be working fine. Go ahead and enter a few sightings by clicking on the link on your thank-you page and entering more data.

With a few sightings entered, let's use App Engine's included **developer console** to confirm that the sightings are in the datastore.

To access the console, enter `http://localhost:8081/_ah/admin` into your web browser's location bar and click on the **List Entities** button to see your data.

In addition to viewing your existing data in the datastore, you can use the console to enter new test data.

The screenshot shows the Google App Engine Datastore Viewer interface. The browser address bar displays `http://localhost:8081/_ah/admin/datastore?kind=Sighting`. The page title is "hfwwg Development Console - Datastore Viewer". The sidebar on the left contains links: Datastore Viewer, Interactive Console, Memcache Viewer, Task Queues, Cron Jobs, XMPP, and Inbound Mail. The main content area shows the "Datastore Viewer" for the "Sighting" entity kind. It includes buttons for "List Entities" and "Create New Entity". Below these is a table of data with columns: Key, ID, Key Name, blow_type, date, email, fin_type, location, name, time, wave_type, and whale_type. The table contains four rows of data. A "Delete" button is visible at the bottom left of the table. Handwritten annotations include: an arrow pointing to the "List Entities" button with the text "In addition to viewing your existing data in the datastore, you can use the console to enter new test data."; an arrow pointing to the "Key" and "ID" columns with the text "App Engine has assigned a 'Key' and an 'ID' to each of your entities, which comes in handy when you need to uniquely identify a sighting."; and an arrow pointing to the table data with the text "There's all the data your entered, which is in a slightly different order than what you might expect. But it's all in there. (App Engine stores your properties in alphabetical order, by name.)".

Key	ID	Key Name	blow_type	date	email	fin_type	location	name	time	wave_type	whale_type
agVoZnd3...	6		Bushy	Jan 5, 2011	eej@hfwwg.org	Triangular	Just off the coast - really close to shore.	Eagle-eyed Jack Jones	8:02am	Breaking	Fin
agVoZnd3...	7		Dense	Wed, Jan 12, 2011	mky@hfwwg.org	Rounded	Off Lighthouse Rock, looking north-east.	Mary Knighty	9:43am	Moderate	Orca
agVoZnd3...	8		Tall	February 20, 2011	jimmy@zep.com	Falcate	Looking west at Smuggler's Cove.	James Page	Early, at dawn.	Moderate	Gray
agVoZnd3...	9		Bushy	2/Mar/2011	rob@zep.com	Rounded	Misty Mountain Bay.	Bob Plant	4:45pm	Flat	Humpback

App Engine has assigned a "Key" and an "ID" to each of your entities, which comes in handy when you need to uniquely identify a sighting.

There's all the data your entered, which is in a slightly different order than what you might expect. But it's all in there. (App Engine stores your properties in alphabetical order, by name.)

Your GAE webapp is now ready for prime time.

Before you deploy it to Google's cloud infrastructure, let's run it by the folk at HFWWG to see if they are happy for their webapp to "go live."

It looks like you're not quite done yet



Is this a **big** change?

You would imagine that it would be. You'll have to create a new entity to hold your registered user login information, and you'll also need *another* form to ask users to provide their registration data (which you'll need to store in the datastore). With that in place, you'll need *yet another form* to ask your users to log in, and then you'll have to come up with a mechanism to *restrict* only registered and logged-in users to view your webapp's pages, assuming you can come up with something robust that will work...?

Or...as this is GAE, you could just switch on authorization.

Sometimes, the tiniest change can make all the difference...

The engineers at Google designed App Engine to deploy on Google's cloud infrastructure. As such, they decided to allow webapps running on GAE to access the *Google Accounts* system.

By switching on **authorization**, you can require users of your webapp to log into their Google account *before* they see your webapp's pages. If a user tries to access your webapp and he isn't not logged in, GAE redirects to the Google Accounts login and registration page. Then, after a successful login, GAE returns the user to your waiting webapp. *How cool is that?*

To switch on authorization, make one small change to your `app.yaml` file:

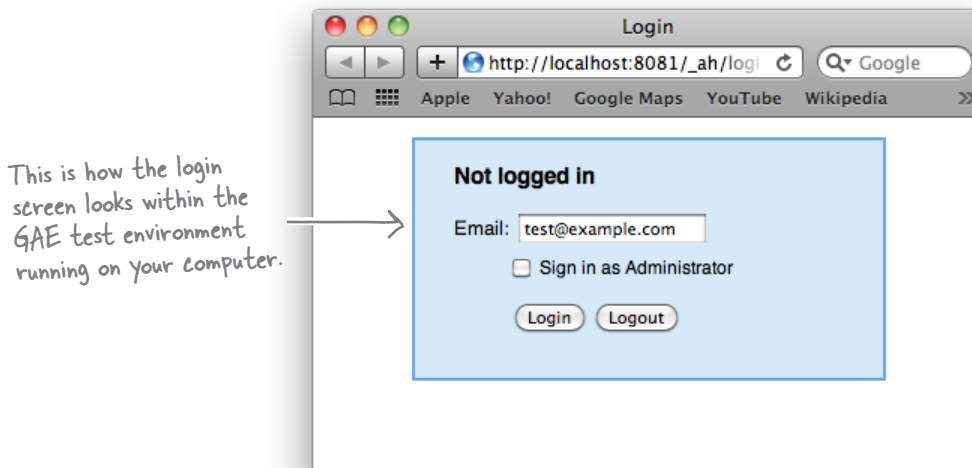
```
application: hfwgapp
version: 1
runtime: python
api_version: 1

handlers:
- url: /static
  static_dir: static

- url: /*
  script: hfwg.py
  login: required
```

That's all there is to it.

Now, when you try to access your webapp, you are asked to log in *before* proceeding.



Capture your user's Google ID, too

Now that your webapp requires your users to log in, let's arrange to capture the user login information as part of the sighting.

Start by adding the following property to your entity's list of attributes in your `hfwwgDB.py` file. Add it right after the `wave_type` property.

Create a new attribute in your "Sighting" class...

```
which_user = db.UserProperty()
```

...and set its property type.

Let's ensure that Django's form validation framework *excludes* this new attribute when generating your HTML form. Within your `hfwwg.py` file, change your `SightingForm` class to look like this:

Make sure Django doesn't include the new attribute in your generated form.

```
class SightingForm(djangoforms.ModelForm):
    class Meta:
        model = hfwwgDB.Sighting
        exclude = ['which_user']
```

Staying within your `hfwwg.py` file, add another import statement near the top of your program:

```
from google.appengine.api import users
```

Import GAE's Google Accounts API.

In your `post()` method, right before you **put** your new sighting to the datastore, add this line of code:

```
new_sighting.which_user = users.get_current_user()
```

When you put your data to the datastore, this code includes the Google ID of the currently logged-in user.

Every time a user adds a sighting to the datastore, GAE ensures that the user's Google Account ID is saved, too. This extra identification information allows the HFWWG to track exactly who reported which sighting, and should (hopefully) cut down on the amount of spam your webapp might attract.

All that's left to do is to deploy your webapp to Google's cloud.

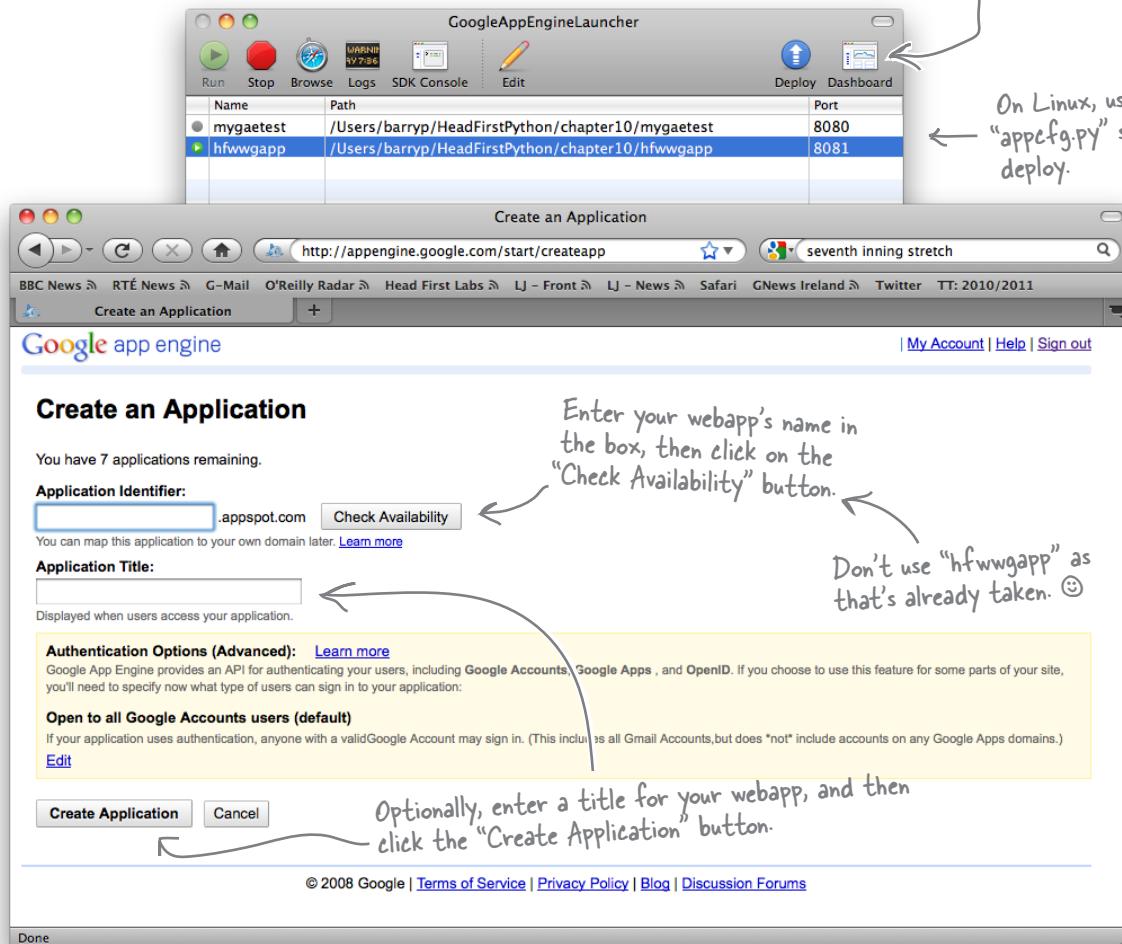
Deploy your webapp to Google's cloud

With your webapp developed and tested locally, you are now ready to deploy to the Google cloud. This is a two-step process: *register* and *upload*.

To register your webapp on the Google cloud, click the **Dashboard** button on the GAE Launcher.

The "Dashboard" button opens your web browser and takes you to the GAE "My Applications" page (after you sign in with your Google ID).

On Linux, use the "appcfg.py" script to deploy.



Assuming all went according to plan and GAE confirmed that your application has been created, all that's left to do is to *deploy*. Return to the GAE Launcher and click on the **Deploy** button. The console displays a bunch of status message while the deployment progresses. If all is well, you'll be told that "appcfg.py has finished with exit code 0".

Your GAE webapp is now ready to run on Google's cloud.



— Test Drive, on Google

Let's take your webapp for a spin on Google's cloud. Open your web browser and surf to a web address that starts with your webapp's name and ends in `.appspot.com`. For the HFWWG webapp, the web address is `http://hfwwgapp.appspot.com`. When you first attempt to go to their webapp, App Engine redirects you to the Google login page.

Google Accounts

google.com https://www.goog

BBC News RTÉ News G-Mail O'Reilly Radar Head First Labs LJ - Front LJ - News Safari GNews Ireland

Google accounts

HeadFirst Whale Watching Group uses Google Accounts for Sign In.

Google is not associated with the contents of **HeadFirst Whale Watching Group** or its owners. If you sign in, Google will share your email address with **HeadFirst Whale Watching Group**, but not your password or any other personal information.

HeadFirst Whale Watching Group may use your email address to personalise your experience on their website.

Sign in with your **Google Account**

Email:
e.g. pat@example.com

Password:

☒ Stay signed in

[Can't access your account?](#)

Don't have a Google Account? [Create an account now](#)

©2010 Google - [Google Home](#) - [Terms of Service](#) - [Privacy Policy](#) - [Help](#)

Done

Provide your Google ID and password, or sign up for a new Google account (if you don't have one already).

After a successful login, your sighting form appears. Go ahead and enter some test data:

The sighting form served from the Google cloud is EXACTLY the same as the form served by your test server.

Return to the <http://appengine.google.com> site to log into the console. The UI is a little different than the test console, but you can use the Datastore Viewer to confirm that your data has been stored correctly.

The screenshot shows the Google App Engine console interface. On the left is a navigation menu with links like Dashboard, Quota Details, Logs, Cron Jobs, Task Queues, Blacklist, Datastore Indexes, **Datastore Viewer** (circled), Datastore Statistics, Blob Viewer, and Administration. The main content area is divided into two parts. The top part, titled 'Report a Possible Sighting', contains a form with the following fields: Name (Jonathon Baldwin), Email (jpi@zep.com), Date (Mar 3, 2011), Time (11:59am), Location (At Bluffer's Beach, about 50m from shore), Fin type (Triangular), Whale type (Gray), Blow type (Tall), and Wave type (Moderate). A 'Submit Sighting' button is at the bottom. The bottom part, titled 'Sighting Entities', shows a table of stored sightings. The table has columns: ID/Name, blow_type, date, email, fin_type, location, name, time, wave_type, whale_type, and which_user. One entity is listed with ID=1. A 'Delete' button is next to the entity row. Handwritten annotations include an arrow pointing from the 'Datastore Viewer' link in the menu to the table, and another arrow pointing from the 'Submit Sighting' button to the table.

Click on this link to see your data as stored in the Google cloud.

ID/Name	blow_type	date	email	fin_type	location	name	time	wave_type	whale_type	which_user
☐ id=1	Tall	Mar 3, 2011	jpi@zep.com	Triangular	At Bluffer's Beach, about 50m from shore.	Jonathon Baldwin	11:59am	Moderate	Gray	hfpypthon

Your HFWWG webapp is deployed!

I'm all packed up and ready for a weekend of whale watching. I can't wait to enter my sightings data online!



This is waaaay cool. Look at that fabulous webapp. This is exactly what we need. Super work!



Lovely... I finally have time to relax, now that I have no more marathon data-entry sessions.



This is professional work.

You've built a *great data-entry webapp* and *deployed* it on Google's cloud. No matter how busy things get, whether there's a handful of sightings per day or tens of thousands, *your webapp can handle the load*, thanks to Google's App Engine. And, best of all, the cash-strapped HFWWG doesn't pay a penny until their sightings activity reaches the level of *millions* of sightings per month!

Did you even notice that you wrote all of your code using Python 2.5?





Your Python Toolbox

You've got Chapter 10 under your belt and you've added more great Python technology to your ever-expanding Python toolbox.

App Engine Lingo

- “Datastore” – the data repository used by Google App Engine to permanently store your data.
- “Entity” – the name used for a “row of data”.
- “Property” – the name used for a “data value”.

- Every App Engine webapp must have a configuration file called `app.yaml`.
- Use the *GAE Launcher* to start, stop, monitor, test, upload, and deploy your webapps.
- App Engine's templating technology is based on the one used in the Django Project.
- App Engine can also use Django's Form Validation Framework.
- Use the `self.response` object to construct a GAE web response.
- Use the `self.request` object to access form data within a GAE webapp.
- When responding to a GET request, implement the required functionality in a `get()` method.
- When responding to a POST request, implement the required functionality in a `post()` method.
- Store data in the App Engine datastore using the `put()` method.

11 dealing with complexity

✧ *Data wrangling* ✧

Once I build up a head of steam, it's not all that hard to keep on running, and running, and running...



It's great when you can apply Python to a specific domain area.

Whether it's *web development*, *database management*, or *mobile apps*, Python helps you **get the job done** by *not* getting in the way of you coding your solution. And then there's the other types of problems: the ones you can't categorize or attach to a domain. Problems that are in themselves so *unique* you have to look at them in a different, highly specific way. Creating **bespoke** software solutions to these type of problems is an area where Python *excels*. In this, your final chapter, you'll *stretch* your Python skills to the limit and solve problems along the way.

What's a good time goal for the next race?

The *Head First Marathon Club* has spent years collecting and collating data on their distance runners. Over time, this data has helped the club produce a large spreadsheet of *pace data* that helps their runners predict their performance over various distances. The spreadsheet is *huge* and runs to 50 columns of tightly packed data.

Let's take a look at the club's data, as well as the way the runners and their coach use it.

The runner selects a distance, say 15K, and we time her over the length of her run.



Here's a portion of the Marathon Club's spreadsheet data.

File Edit View Insert Format Form Tools Help							
Formula: V02							
	A	B	C	D	E	F	
1	V02	84.8	82.9	81.1	79.3	77.5	75.8
2	2mi	8:00	8:10	8:21	8:33	8:44	8:56
3	5k	12:49	13:06	13:24	13:42	14:00	14:19
4	5mi	21:46	21:48	22:17	22:47	23:18	23:50
5	10k	26:54	27:30	28:08	28:45	29:24	30:04
6	15k	41:31	42:27	43:24	44:23	45:23	46:24
7	10mi	44:46	45:46	46:48	47:51	48:56	50:02
8	20k	56:29	57:45	59:03	1:00:23	1:01:45	1:03:08
9	13.1mi	59:49	1:01:09	1:02:32	1:03:56	1:05:21	1:06:51
10	25k	1:11:43	1:13:20	1:14:59	1:16:40	1:18:24	1:20:10
11	30k	1:27:10	1:19:08	1:31:08	1:33:11	1:35:17	1:37:26
12	Marathon	2:05:34	2:08:24	2:11:17	2:14:15	2:17:16	2:20:21
13							

The timed distance is 15km.

The predicted marathon goal.

Yes, she's wicked fast!

I run the 15K in 45:01. My coach looks up the closest match for my time along the 15K row.

This benchmark allows me to look up or down the column to select a target time for any other distance, such as a marathon, for example.



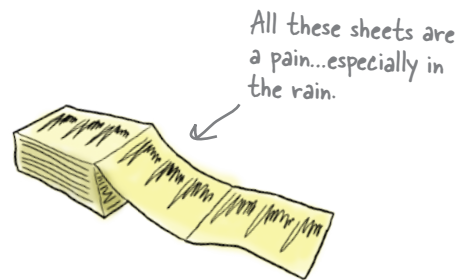
This spreadsheet is a little intimidating...but don't worry. You'll sort it out soon.

G	H	I	J	K	L	M	N	O	P
	74.2	72.5	70.9	69.4	67.9	66.4	64.9	63.5	62.1
	9:08	9:20	9:33	9:46	9:59	10:13	10:26	10:41	10:55
	14:38	14:58	15:18	15:39	16:00	16:22	16:44	17:06	17:28
	24:22	24:55	25:28	26:03	26:38	27:14	27:51	28:28	29:05
	30:45	31:26	32:09	32:52	33:36	34:22	35:08	35:56	36:43
	47:27	48:31	49:36	50:43	51:52	53:02	54:14	55:27	56:41
	51:09	52:18	53:29	54:41	55:55	57:11	58:28	59:47	61:07
	1:04:33	1:06:00	1:07:29	1:09:01	1:10:34	1:12:09	1:13:46	1:15:26	1:17:07
	1:08:21	1:09:53	1:11:28	1:13:04	1:14:43	1:16:24	1:18:07	1:19:52	1:21:38
	1:21:58	1:23:49	1:25:49	1:27:37	1:29:36	1:31:37	1:33:40	1:35:47	1:37:55
	1:39:37	1:41:52	1:44:09	1:46:30	1:48:54	1:51:21	1:53:51	1:56:25	1:59:00
	2:23:31	2:26:44	2:30:02	2:33:25	2:36:52	2:40:24	2:44:00	2:47:42	2:51:28

So...what's the problem?



At the moment, we print our data onto multiple sheets of paper, which we carry with us. Most of the time, this works fine. But when it rains or gets really windy, our pages are either soaked through or they end up all over the place.



Not to mention: forgetting the sheets, keeping the sheets up to date, and having to flip back and forth through the sheets looking for a closest match.

Of course, word of your newly acquired Python programming skills is getting around, especially among the running crowd. Ideally, the Marathon Club needs an Android app that can be loaded onto a bunch of phones and carried in each coach's pocket. The app needs to automate the lookup and distance predictions.

Are you up to the challenge? Do you think you can help?

Start with the data

For now, let's not worry about creating the Android app; you'll get to that soon enough. Instead, let's solve the central **data wrangling problem** and then, when you have a working solution, we'll worry about *porting* your solution to Android. We'll begin by getting the data into a format you can easily process with Python.

Most spreadsheet programs can export data to the widely used CSV format. The club has done this for you and created a file called `PaceData.csv`, which includes the data for each of the rows from the original the spreadsheet.

Here's a sample of the raw data from the start of the CSV:

The first line of data is the column headings from the spreadsheet. They look like numbers but are actually headings that represent estimated maximal oxygen consumption (or *VO2 Max*, in ml/kg-min) for the race times in each column. Because they have no affect on the timing data, we'll just treat them as headings.

```
VO2,84.8,82.9,81.1,79.3,77.5,75.8,74.2,72.5,70.9,69.4,67.9,66.4,64.9,63.5,62.1,60.7,59.4,58.1,56.8,55.2mi,8:00,8:10,8:21,8:33,8:44,8:56,9:08,9:20,9:33,9:46,9:59,10:13,10:26,10:41,10:55,11:10,11:25,11:40,15k,12:49,13:06,13:24,13:42,14:00,14:19,14:38,14:58,15:18,15:39,16:00,16:22,16:44,17:06,17:30,17:53,18:
```

The first value on each of the rest of the lines is the timed distance or row label.

The rest of each line is a list of recorded run times.

Sharpen your pencil



You somehow have to model the data from the CSV file in your Python program. Can you think of a data structure that might help here? Justify your selection.

.....

.....

.....

.....

.....



Grab a copy of `PaceData.csv` from this book's support website.

Sharpen your pencil Solution

You somehow have to model the data from the CSV file in your Python program. You were to think of a data structure that might help here? You were also to justify your selection.

Ummm... there's lots to think about here.

The list of headings can be stored in a **LIST**.

The list of times from each row can also be stored in a **LIST**, but they also need to be associated with the headings in the very first row of data, so maybe a **DICTIONARY** is what's needed here?

Maybe it's some **COMBINATION** of the two?!?

Take another look at the data

The first row of data in the CSV file is the **column headings**, with the very first value on the line, the **V02** string, being redundant (it won't ever be used in this version of the app). The rest of the first line's data are headings *associated* with the time values in each of the columns.

Of course, the data in the columns is *also* associated with each row, which is identified by a **row label** in the first column, such as **2mi**, **5k**, and so on.

Let's look at the data in the CSV file again, which has been reformatted to help highlight the associations.

This string is redundant.

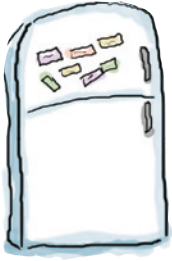
The column headings are on the first line.

V02,	84.8,	82.9	81.1,	79.3,	77.5,	75.8,	74.2
2mi,	8:00,	8:10,	8:21,	8:35,	8:44,	8:56,	9:04,
5k,	12:49,	13:06,	13:24,	13:42,	14:00,	14:19,	14:38

The row labels are unique to each row of times.

The times on each row are associated with their row label, but ALSO with a column heading.

But can we capture all these associations in code?



Marathon Magnets

Here's some code that reads the raw data from the CSV data file. The column headings from the first line are loaded into a list called `column_headings`. The rest of the data (all the rows of times) are loaded into a dictionary called `row_data`, with each row of data keyed with the **row label string** from the start of each line. Of course, as luck would have it, someone was cleaning the fridge door, and they've left a bunch of magnets on the floor. See if you can arrange the magnets into their correct order.

What needs
to go here?

```
with open('PaceData.csv') as paces:
```

Process the
"column_headings"
list here.

```
    for each_line in paces:
```

Process
"row_data"
here.

```
        num_cols = len(column_headings)
        print(num_cols, end=' -> ')
        print(column_headings)
```

```
        num_2mi = len(row_data['2mi'])
        print(num_2mi, end=' -> ')
        print(row_data['2mi'])
```

```
        num_Marathon = len(row_data['Marathon'])
        print(num_Marathon, end=' -> ')
        print(row_data['Marathon'])
```

With the data
loaded, this code
lets you check if it's
all OK.

```
row = each_line.strip().split(',')
```

```
column_headings =
```

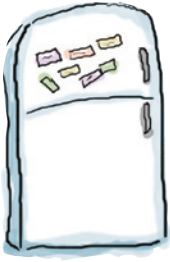
```
row_data[row_label] = row
```

```
row_label = row.pop(0)
```

```
row_data = {}
```

```
paces.readline().strip().split(',')
```

```
column_headings.pop(0)
```



Marathon Magnets Solution

Here's some code that reads the raw data from the CSV data file. The column headings from the first line are loaded into a list called `column_headings`. The rest of the data (all the rows of times) are loaded into a dictionary called `row_data`, with each row of data keyed with the **row label string** from the start of each line. Of course, as luck would have it, someone was cleaning the fridge door, and they've left a bunch of magnets on the floor. You were to see if you could arrange the magnets into their correct order.

```

row_data = {}
with open('PaceData.csv') as paces:
    column_headings = paces.readline().strip().split(',')
    column_headings.pop(0)
    for each_line in paces:
        row = each_line.strip().split(',')
        row_label = row.pop(0)
        row_data[row_label] = row

num_cols = len(column_headings)
print(num_cols, end=' -> ')
print(column_headings)

num_2mi = len(row_data['2mi'])
print(num_2mi, end=' -> ')
print(row_data['2mi'])

num_Marathon = len(row_data['Marathon'])
print(num_Marathon, end=' -> ')
print(row_data['Marathon'])

```

Annotations:

- `row_data = {}`: You need to be sure to create an empty dictionary for the row times.
- `with open('PaceData.csv') as paces:`: Create the column headings from the first line of data.
- `column_headings = paces.readline().strip().split(',')`: Read a line from the file, strip it of unwanted whitespace, and then split the line on comma.
- `column_headings.pop(0)`: Delete the first heading, the "VO2" string—you don't need it.
- `for each_line in paces:`: Process the rest of the file.
- `row = each_line.strip().split(',')`: It's the same deal here: take the line, strip it, and then split on comma.
- `row_label = row.pop(0)`: Extract the row label.
- `row_data[row_label] = row`: Use the row label together with the rest of the line's data to update the dictionary.



Test Drive

Load your code into IDLE and, with the CSV in the same folder as your code, run it to see what you get on screen.

Your code
in IDLE.

```
marathon.py - /Users/barryp/HeadFirstPython/chapter11/marathon.py

row_data = {}

with open('PaceData.csv') as paces:

    column_headings = paces.readline().strip().split(',')
    column_headings.pop(0)

    for each_line in paces:
        row = each_line.strip().split(',')
        row_label = row.pop(0)
        row_data[row_label] = row

num_cols = len(column_headings)
print(num_cols, end=' -> ')
print(column_headings)

num_2mi = len(row_data['2mi'])
print(num_2mi, end=' -> ')
```

The output confirms
that each row of
data has 50 data
items.

The column
headings

The "2mi"
row of
data

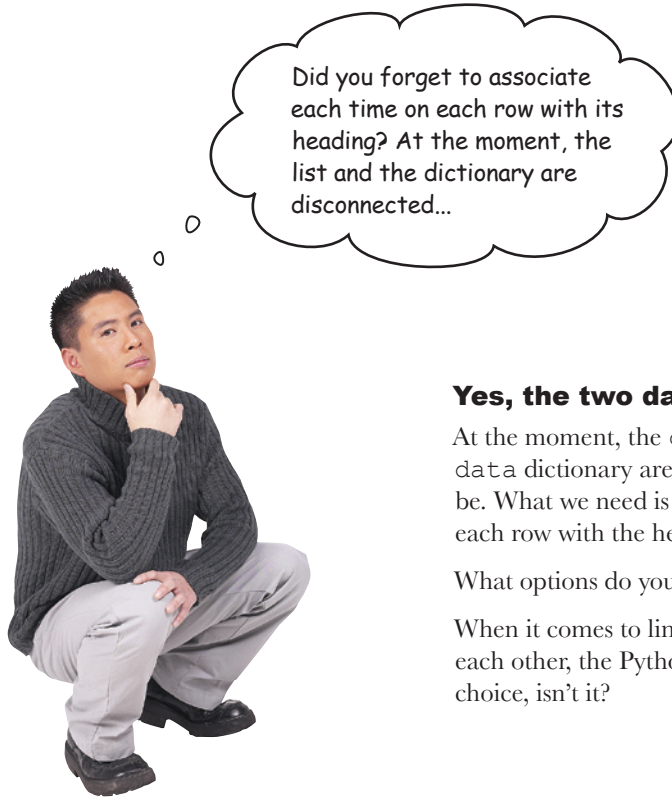
The
"Marathon"
row of
data

```
Python Shell
Python 3.1.2 (r312:79360M, Mar 24 2010, 01:33:18)
[GCC 4.0.1 (Apple Inc. build 5493)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
50 -> ['84.8', '82.9', '81.1', '79.3', '77.5', '75.8', '74.2', '72.5', '70.9', '69.4', '67.9', '66.4', '64.9', '63.5', '62.1', '60.7', '59.4', '58.1', '56.8', '55.5', '54.3', '53.1', '52', '50.8', '49.7', '48.6', '47.5', '46.5', '45.5', '44.5', '43.5', '42.5', '41.6', '40.7', '39.8', '38.9', '38', '37.2', '36.4', '35.6', '34.5', '34', '33.3', '32.6', '31.8', '31.1', '30.5', '29.8', '29.1', '28.5']
50 -> ['8:00', '8:10', '8:21', '8:33', '8:44', '8:56', '9:08', '9:20', '9:33', '9:46', '9:59', '10:13', '10:26', '10:41', '10:55', '11:10', '11:25', '11:40', '11:56', '12:12', '12:29', '12:45', '13:03', '13:20', '13:38', '13:57', '14:16', '14:35', '14:54', '15:15', '15:35', '15:56', '16:18', '16:40', '17:02', '17:25', '17:49', '18:13', '18:38', '19:03', '19:28', '19:55', '20:22', '20:49', '21:17', '21:46', '22:15', '22:45', '23:16', '23:48']
50 -> ['2:05:34', '2:08:24', '2:11:17', '2:14:15', '2:17:16', '2:20:21', '2:23:31', '2:26:44', '2:30:02', '2:33:25', '2:36:52', '2:40:24', '2:44:00', '2:47:42', '2:51:28', '2:55:20', '2:59:16', '3:03:18', '3:07:26', '3:11:39', '3:15:58', '3:20:22', '3:24:53', '3:29:29', '3:34:12', '3:39:01', '3:43:57', '3:48:59', '3:54:09', '3:59:25', '4:04:48', '4:10:18', '4:15:56', '4:21:42', '4:27:35', '4:33:36', '4:39:46', '4:46:04', '4:52:30', '4:59:05', '5:05:48', '5:12:41', '5:19:43', '5:26:55', '5:34:16', '5:41:48', '5:49:29', '5:57:21', '6:05:23', '6:13:37']
>>>
```

Ln: 9 Col: 4

That's a great start: you've managed to read the data from the CSV and put the headings into a list and the data into a dictionary.

What's next?



Did you forget to associate each time on each row with its heading? At the moment, the list and the dictionary are disconnected...

Yes, the two data structures should be linked.

At the moment, the `column_headings` list and the `row_data` dictionary are not linked in any way, and they need to be. What we need is some way to *connect* each of the times in each row with the heading that tops their column of data.

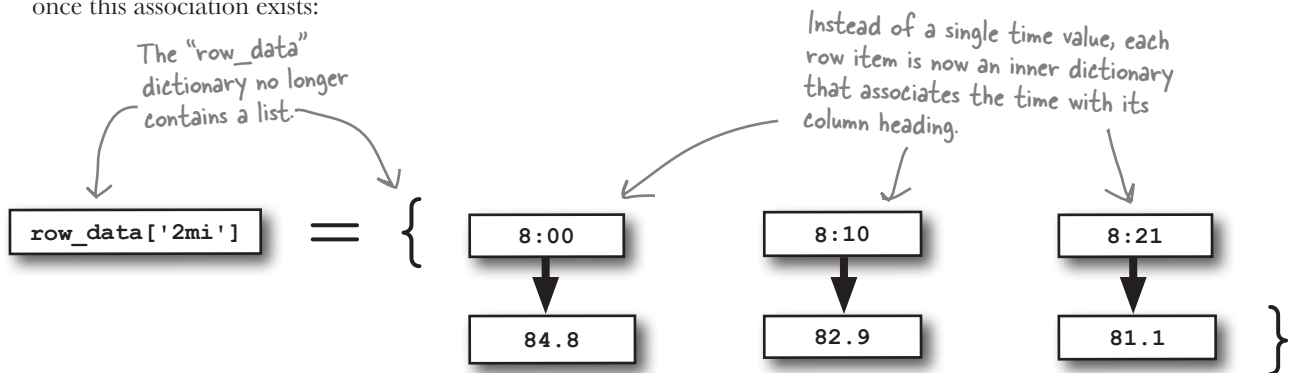
What options do you have here?

When it comes to linking (or associating) two data items with each other, the Python dictionary is the data structure of choice, isn't it?

Store each time as a dictionary

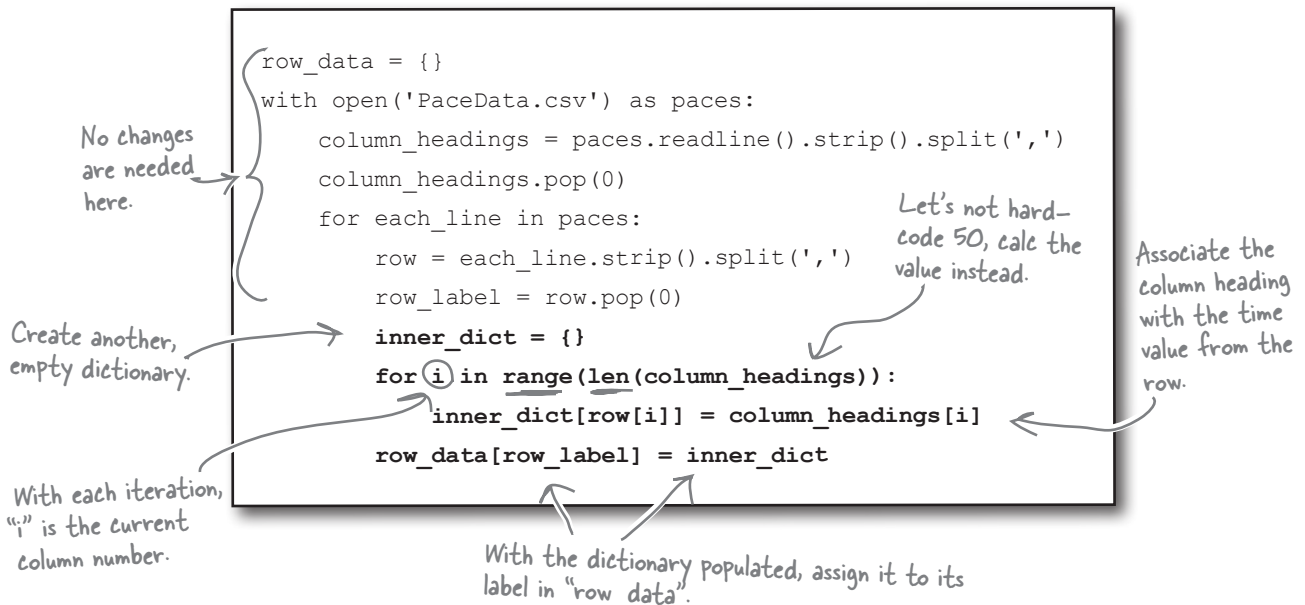
Rather than simply storing each time in the `row_data` dictionary as a number, let's store the data as *as a dictionary*, with the *key* set to the time and the *value* set to the column heading. That way, you can quickly and easily determine for any time which column it is associated with, right?

Here's a portion of what the data structure looks like in Python's memory once this association exists:



All you need to do is work out how to populate the inner dictionary with the row data and the associated columns headings...and you'll have all the data you need.

The trick in creating the data structure is to realize that each row, *including the column headings*, are of a **fixed size**: 50 items. Knowing this, it's not much work to create the dictionary you need:





An IDLE Session

Go ahead and add the extra dictionary populating code to your program. Let's remove all of those `print()` statements from the end of your program, because you'll use the IDLE shell to test your code. Run the code by pressing `F5` or by selecting the Run Module option from the Run menu. Use the `dir()` BIF to confirm that your program code executed and that a collection of variables have been created in Python's namespace:

```
>>> dir()
['_builtins_', '__doc__', '__name__', '__package__', 'column_headings', 'each_line', 'i',
'inner_dict', 'paces', 'row', 'row_data', 'row_label']
```

← All of your code's variables exist.

V02	84.8	82.9	81.1	79.3	77.5
2mi	8:00	8:10	8:21	8:33	8:44
5k	12:49	13:06	13:24	13:42	14:00
5mi	21:19	21:48	22:17	22:47	23:18
10k	26:54	27:30	28:08	28:45	29:24
15k	41:31	42:27	43:24	44:23	45:23
10mi	44:46	45:46	46:46	47:51	48:56
20k	56:29	57:45	59:03	1:00:23	1:01:45
13.1mi	59:49	1:01:09	1:02:32	1:03:56	1:05:23
25k	1:11:43	1:13:20	1:14:59	1:16:40	1:18:24
30k	1:27:10	1:19:08	1:31:08	1:33:11	1:35:17
Marathon	2:05:34	2:08:24	2:11:17	2:14:15	2:17:16

Take another look at (part of) the spreadsheet data file above, and let's try and find the column heading associated with the **43:24** time on the **15k** row. Let's then use the column heading to find the predicted time for a **20k** run:

```
>>> column_heading = row_data['15k']['43:24']
```

```
>>> column_heading
```

```
'81.1'
```

← The associated column heading is correctly identified as "81.1".

```
>>> prediction = [k for k in row_data['20k'].keys() if row_data['20k'][k] == column_heading]
```

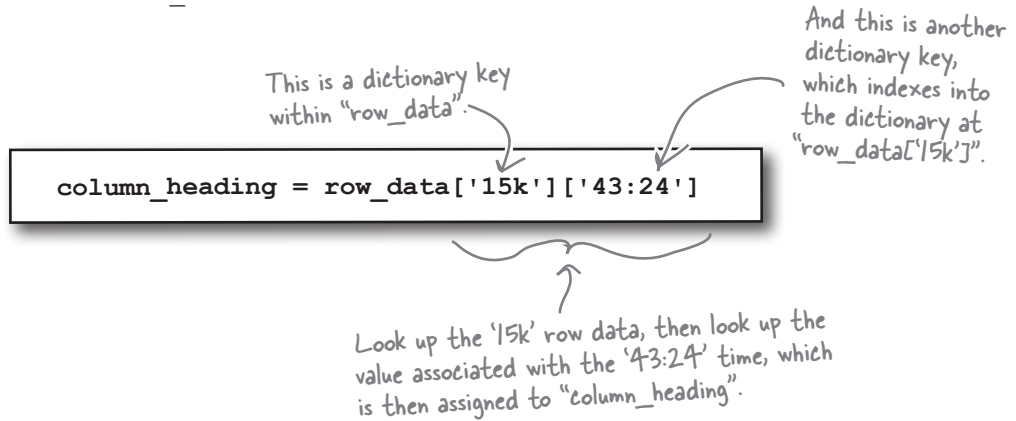
```
>>> prediction
```

```
['59:03']
```

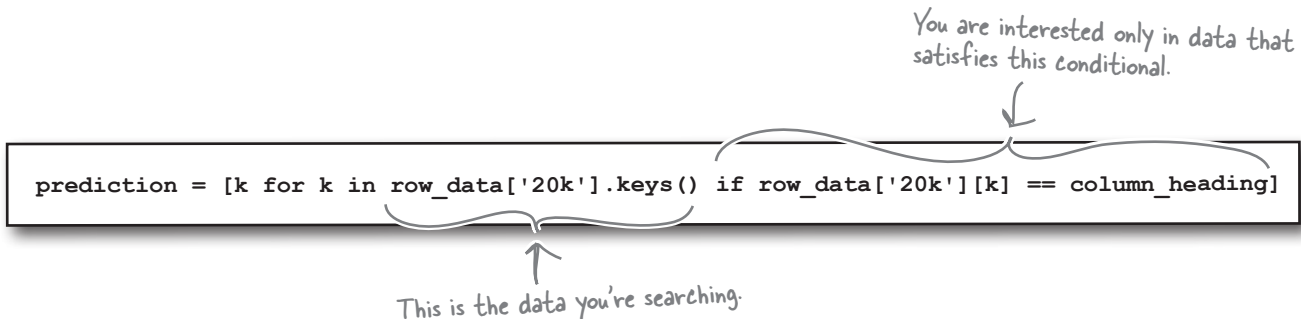
← A time of "59:03" is correctly predicted, too.

Dissect the prediction code

Let's take a moment to review what just happened at the bottom of the IDLE Session from the last page. This line of code is a *double-dictionary lookup* on the dictionary-of-dictionaries stored in `row_data`:



Working out the predicted time in the **20k** row of data involves **finding the key** in the row's dictionary whose *value* is set to the just-discovered value stored in `column_heading`.



A *conditional list comprehension* is put to good use here. Recall that the list comprehension syntax is a shorthand notation for a **for** loop. The loop searches through the data in the list of keys associated with the dictionary stored at `row_data['20k']`. If the value associated with the key (in `k`) is the same as `column_heading`, the value of `k` is added to the comprehension's results, which are then assigned to a new list call `prediction`.

There's really an awful lot going on in that comprehension.



Sweet mother of all things
Python! What's going on
here? I think my brain is
going to explode...

Don't let the list comprehension put you off.

Recall that you can always rewrite a list comprehension using an equivalent **for** loop...

Ummm...now there's an idea.



Rewrite each of the list comprehensions on this page to use a **for** loop.

```
times = [t for t in row_data['Marathon'].keys()]
```

.....


```
headings = [h for h in sorted(row_data['10mi'].values(), reverse=True)]
```

.....


```
time = [t for t in row_data['20k'].keys() if row_data['20k'][t] == '79.3']
```

.....

Sharpen your pencil

Solution

You were to rewrite each of the list comprehensions to use a **for** loop.

```
times = [t for t in row_data['Marathon'].keys()]
```

Start with an empty list.

→ `times = []`

`for each_t in row_data['Marathon'].keys():`

Turn the dictionary's keys into a list.

`times.append(each_t)`

With each iteration, append the key (which is a time value) onto the "times" list.

```
headings = [h for h in sorted(row_data['10mi'].values(), reverse=True)]
```

Start with an empty list.

→ `headings = []`

Turn the dictionary's values into a list..

`for each_h in sorted(row_data['10mi'].values(), reverse=True):`

`headings.append(each_h)`

...being sure to sort the values in reverse order (biggest first).

With each iteration, append the value (which is a column heading) onto the "times" list.

```
time = [t for t in row_data['20k'].keys() if row_data['20k'][t] == '79.3']
```

Start with an empty list.

→ `time = []`

Turn the dictionary's keys into a list.

`for each_t in row_data['20k'].keys():`

`if row_data['20k'][each_t] == '79.3':`

`time.append(each_t)`

With each iteration, check to see if the column heading (the value part of the dictionary) equals "79.3" and if it does, append the time to the list.

There's a definite pattern emerging here. 😊

Get input from your user

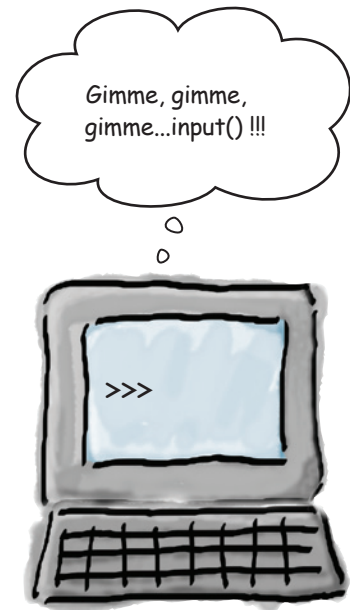
Now that you have your data within a Python data structure, it's time to ask your user what it is they are looking for.

Specifically, you need to know three things: the *distance* run, the *time* recorded, and the distance a *prediction* is required for.

When you get to move your app onto Android, you can use a nice graphical dialog to ask your user for input, but for now, let's quickly create a text-based user interface, which will allow you to develop and test the rest of the functionality required from your application. When you're done, you'll create the Android app.

Use input() for input

Python has the `input()` BIF that can help here, which is used to display a prompt on screen, and then accept keyboard input, returning what was entered as a string to your code.



An IDLE Session

Using the `input()` BIF is best demonstrated with some examples:

```
>>> res = input('What is your favorite programming language: ')
```

```
What is your favorite programming language: Python
```

```
>>> res
```

```
'Python'
```

The entered data is assigned to "res" and it's a **STRING**.

Provide the prompt to display to your user.

The `input()` BIF returns a string, which has been stripped of any trailing newline character, which would typically be included at the end of any input string. It is important to note that any input is returned as a string, regardless of what type of data you think you might be entering:

The entered data is assigned to "age" and it's a string, even though you might want to treat it like it's a number.

```
>>> age = input('What is your age: ')
```

```
What is your age: 21
```

```
>>> age
```

```
'21'
```

```
>>> int(age)
```

```
21
```

Convert the input to the type you need **BEFORE** using the data.

[Editor's note: Yeah... dream on, Paul. 😊]

Getting input raises an issue...

It's not hard to use `input()` to get the, um, er...input you need. Here's your code from earlier with three calls to `input()` added to interact with your user.

```

marathon.py - /Users/barryp/HeadFirstPython/chapter11/marathon.py

row_data = {}

with open('PaceData.csv') as paces:

    column_headings = paces.readline().strip().split(',')
    column_headings.pop(0)

    for each_line in paces:
        row = each_line.strip().split(',')
        row_label = row.pop(0)
        inner_dict = {}
        for i in range(len(column_headings)):
            inner_dict[row[i]] = column_headings[i]
        row_data[row_label] = inner_dict

distance_run = input('Enter the distance attempted: ')
recorded_time = input('Enter the recorded time: ')
predicted_distance = input('Enter the distance you want a prediction for: ')

```

There's nothing to this, as user-interaction with "input()" doesn't get much easier than this..

Ln: 20 Col: 0

When your program runs, your user enters some data, and *look what happens*:

```

Python Shell

>>> ===== RESTART =====
>>>
Enter the distance attempted: 20k
Enter the recorded time: 59:59
Enter the distance you want a prediction for: Marathon
>>>
>>>
>>> row_data[distance_run][recorded_time]
Traceback (most recent call last):
  File "<pyshell#76>", line 1, in <module>
    row_data[distance_run][recorded_time]
KeyError: '59:59'
>>>

```

A "KeyError" exception has been raised...but why?

Ln: 205 Col: 4

If it's not in the dictionary, it can't be found.

The data in the `row_data` dictionary originally comes from the spreadsheet and is read into your program from the CSV file.

If the data value entered into the `recorded_time` variable is in the dictionary, things are going to be fine, because there's a *match*. However, if the data entered into the `recorded_time` variable *doesn't* match anything in the dictionary, you'll get a `KeyError`.

But how is this “problem” handled during training?



The entered time for a 20k run (59:59) falls between these two values on the pace sheet.

10k	26:54	27:30	28:08	28:45	29:24
15k	41:31	42:27	43:24	44:23	45:23
10mi	44:46	45:46	46:48	47:51	48:56
20k	56:29	57:45	59:03	1:00:23	1:01:45
13.1mi	59:49	1:01:09	1:02:32	1:03:56	1:05:23
25k	1:11:43	1:13:20	1:14:59	1:16:40	1:18:24
30k	1:27:10	1:19:08	1:31:08	1:33:11	1:35:17
Marathon	2:05:34	2:08:24	2:11:17	2:14:15	2:17:16

Search for the closest match

All you need to do is search the row of data for the closest match, right?
And guess what? The Head First Code Review Team think they have some functions that might help here.

There's nothing better than sharing our code with our fellow Python programmers. Check out our "find_it" module.

This code is in a file called "find_it.py" and you can download a copy from this book's support website.



```

find_it.py - /Users/barryp/HeadFirstPython/chapter11/find_it.py

def find_closest(look_for, target_data):
    def whats_the_difference(first, second):
        if first == second:
            return(0)
        elif first > second:
            return(first - second)
        else:
            return(second - first)

    max_diff = 9999999
    for each_thing in target_data:
        diff = whats_the_difference(each_thing, look_for)
        if diff == 0:
            found_it = each_thing
            break
        elif diff < max_diff:
            max_diff = diff
            found_it = each_thing
    return(found_it)

```

The "find_closest" function does a simple linear search, returning the value in "target_data" that most closely matches the "look_for" argument.

Here's an example of a nested function, which is allowed in Python. Given two values, this function returns the difference between them.

This may not be the most efficient search code ever written, but it works.



An IDLE Session

Let's test the `find_it.py` module to try and determine if it meets the requirements of your application. Load the module into IDLE and then press **F5** or choose Run Module from the Run menu:

```
>>> find_closest(3.3, [1.5, 2.5, 4.5, 5.2, 6])
2.5
>>> find_closest(3, [1, 5, 6])
1
>>> find_closest(3, [1, 3, 4, 6])
3
>>> find_closest(3.6, [1.5, 2.5, 4.5, 5.2, 6])
4.5
>>> find_closest(3, [1, 4, 6])
4
>>> find_closest(2.6, [1.5, 2.5, 4.5, 5.2, 6])
2.5
```

Given a value to look for and some target data, the "find_closest" function seems to be doing the trick.

Let's try it with some of data that more closely resembles your CSV data:

```
>>> find_closest('59:59', ['56:29', '57:45', '59:03', '1:00:23', '1:01:45'])
Traceback (most recent call last):
  File "<pyshell#23>", line 1, in <module>
    find_closest('59:59', ['56:29', '57:45', '59:03', '1:00:23', '1:01:45'])
  File "/Users/barryp/HeadFirstPython/chapter11/find_it.py", line 15, in find_closest
    if diff == 0:
  File "/Users/barryp/HeadFirstPython/chapter11/find_it.py", line 11, in whats_the_difference
    TypeError: unsupported operand type(s) for -: 'str' and 'str'
```

Yikes! Something's seriously broken here.



What do you think has gone wrong here? Why does the `find_closest()` function crash when asked to work with data from your CSV file?

The trouble is with time

The data in your CSV file is a *representation* of timing values. Rather than actual numbers, the values in the CSV are **strings**. This is great for you, because *you* understand what the representation means. Python, on the other hand, sees the data only as strings.

When you send your data to the `find_closest()` function, Python *attempts to treat your strings as numbers* and chaos ensues. What might work would be to convert the time-strings into numbers. But how?



The time-to-seconds-to-time module

The Head First Code Review Team's generosity knows no bounds. Sure enough, their rather strangely name `tm2secs2tm.py` module looks like it might help.

Here's the guy's
"tm2secs2tm.py" module.

Grab a copy of this code from
this book's support website.

```
tm2secs2tm.py - /Users/barryp/HeadFirstPython/chapter11/tm2secs2tm.py

import time

def format_time(time_string):
    tlen = len(time_string)
    if tlen < 3:
        original_format = '%S'
    elif tlen < 6:
        original_format = '%M:%S'
    else:
        original_format = '%H:%M:%S'
    time_string = time.strftime('%H:%M:%S', time.strptime(time_string, original_format))
    return time_string

def time2secs(time_string):
    time_string = format_time(time_string)
    (hours, mins, secs) = time_string.split(':')
    seconds = int(secs) + (int(mins)*60) + (int(hours)*60*60)
    return seconds

def secs2time(seconds):
    return time.strftime('%H:%M:%S', time.gmtime(seconds))
```

This function ensures that all times are formatted in "HH:MM:SS" format. This helps keep things simple when doing conversions to seconds.

Given a "time string", convert it to a value in seconds.

Convert a value in seconds to a "time string".

Ln: 24 Col: 0



Exercise

Now that you have the `tm2secs2tm.py` and `find_it.py` modules, let's create a function that uses the facilities provided by these modules to solve your searching problem. Your new function, called `find_nearest_time()`, takes two arguments: the time to look for and a list of times to search. The function returns the closest time found as a string:

The code you
need has been
started for
you.

```
from find_it import find_closest
from tm2secs2tm import time2secs, secs2time

def find_nearest_time(look_for, target_data):
```

Unlike in the previous
chapter, it is possible
to do what you need
to do here in only
four lines of code.

```
.....
.....
.....
.....
```



Exercise Solution

Now that you have the `tm2secs2tm.py` and `find_it.py` modules, you were to create a function that uses the facilities provided by these modules to solve your searching problem. Your new function, called `find_nearest_time()`, takes two arguments: the time to look for and a list of times to search. The function returns the closest time found as a string:

```

import the team's code. {
    from find_it import find_closest
    from tm2secs2tm import time2secs, secs2time

    def find_nearest_time(look_for, target_data):
        Convert the time string
        you are looking for into its
        equivalent value in seconds. → what = time2secs(look_for)

        where = [time2secs(t) for t in target_data] ← Convert the lines of time
        strings into seconds.

        Call "find_closest()",
        supplying the converted data. → res = find_closest(what, where)

        return(secs2time(res)) ← Return the closest match to the calling
        code, after converting it back to a time
        string.

```

The function takes two arguments, a time string and a list of time strings.



An IDLE Session

Let's try out your code at the IDLE shell to see if your time "problems" have been resolved:

Here's some of your pace data. Let's work with data from the "20k" row.

10k	26:54	27:30	28:08	28:45	29:24
15k	41:31	42:27	43:24	44:23	45:23
40k	44:46	45:46	46:48	47:51	48:56
20k	56:29	57:45	59:03	1:00:23	1:01:45
13.1mi	59:49	1:01:09	1:02:32	1:03:56	1:05:23
25k	1:11:43	1:13:20	1:14:59	1:16:40	1:18:24
30k	1:27:10	1:19:08	1:31:08	1:33:11	1:35:17
Marathon	2:05:34	2:08:24	2:11:17	2:14:15	2:17:16

```

>>> find_nearest_time('59:59', ['56:29', '57:45', '59:03', '1:00:23', '1:01:45'])
'01:00:23'

>>> find_nearest_time('1:01:01', ['56:29', '57:45', '59:03', '1:00:23', '1:01:45'])
'01:00:23'

>>> find_nearest_time('1:02:01', ['56:29', '57:45', '59:03', '1:00:23', '1:01:45'])
'01:01:45'

>>> find_nearest_time('57:06', ['56:29', '57:45', '59:03', '1:00:23', '1:01:45'])
'00:56:29'

```

Great! This appears to be working fine.



Test Drive

With all this code available to you, it's an easy exercise to put it all together in your program and produce a complete solution to the Marathon Club's prediction problem. Let's take it for a test run.

```
marathon.py - /Users/barryp/HeadFirstPython/chapter11/marathon.py

from find_it import find_closest
from tm2secs2tm import time2secs, secs2time

def find_nearest_time(look_for, target_data):
    what = time2secs(look_for)
    where = [time2secs(t) for t in target_data]
    res = find_closest(what, where)
    return(secs2time(res))

row_data = {}

with open('PaceData.csv') as paces:
    column_headings = paces.readline().strip().split(',')
    column_headings.pop(0)

    for each_line in paces:
        row = each_line.strip().split(',')
        row_label = row.pop(0)
        inner_dict = {}
        for i in range(len(column_headings)):
            inner_dict[row[i]] = column_headings[i]
        row_data[row_label] = inner_dict

distance_run = input('Enter the distance attempted: ')
recorded_time = input('Enter the recorded time: ')
predicted_distance = input('Enter the distance you want a prediction for: ')

closest_time = find_nearest_time(recorded_time, row_data[distance_run])
closest_column_heading = row_data[distance_run][closest_time]

prediction = [k for k in row_data[predicted_distance].keys()
              if row_data[predicted_distance][k] == closest_column_heading]

print('The predicted time running ' + predicted_distance + ' is: ' + prediction[0] + '.')
```

This code is used "as is".

Find the nearest time within the data.

Extract the column heading.

Search for a predicted time at the desired distance and display it on screen.

Ln: 37 Col: 0

```
Python Shell

>>> ===== RESTART =====
>>>
Enter the distance attempted: 20k
Enter the recorded time: 59:59
Enter the distance you want a prediction for: Marathon
Traceback (most recent call last):
  File "/Users/barryp/HeadFirstPython/chapter11/marathon.py", line 31, in <module>
    closest_column_heading = row_data[distance_run][closest_time]
  File "/Users/barryp/HeadFirstPython/chapter11/marathon.py", line 31, in <module>
    closest_column_heading = row_data[distance_run][closest_time]
KeyError: '01:00:23'

>>>
```

Try out your program with the same data input from earlier.

Another "KeyError"...

Ln: 13 Col: 4

After all that, you're getting the same error as before. Bummer.

The trouble is still with time...

Or, to be more precise, with how the `tm2secs2tm.py` module formats time strings. Take another look at the results from the previous IDLE Session. Do you notice anything strange about the results returned by the call to the `find_nearest_time()` function?

All the returned times use the "HH:MM:SS" format.

```
>>> find_nearest_time('59:59', ['56:29', '57:45', '59:03', '1:00:23', '1:01:45'])
'01 00:23'
>>> find_nearest_time('1:01:01', ['56:29', '57:45', '59:03', '1:00:23', '1:01:45'])
'01 00:23'
>>> find_nearest_time('1:02:01', ['56:29', '57:45', '59:03', '1:00:23', '1:01:45'])
'01 01:45'
>>> find_nearest_time('57:06', ['56:29', '57:45', '59:03', '1:00:23', '1:01:45'])
'00 56:29'
```

When your code takes one of these returned values and tries to index into your dictionary, there's no match found, because your dictionary's keys do **not** confirm to the HH:MM:SS format. The solution to this problem is to ensure that *every time* you use a time-string in your code, make sure it's in HH:MM:SS format:

```
from find_it import find_closest
from tm2secs2tm import time2secs, secs2time, format_time

def find_nearest_time(look_for, target_data):
    what = time2secs(look_for)
    where = [time2secs(t) for t in target_data]
    res = find_closest(what, where)
    return(secs2time(res))

row_data = {}

with open('PaceData.csv') as paces:
    column_headings = paces.readline().strip().split(',')
    column_headings.pop(0)

    for each_line in paces:
        row = each_line.strip().split(',')
        row_label = row.pop(0)
        inner_dict = {}
        for i in range(len(column_headings)):
            inner_dict[format_time(row[i])] = column_headings[i]
        row_data[row_label] = inner_dict

distance_run = input('Enter the distance attempted: ')
recorded_time = input('Enter the recorded time: ')
predicted_distance = input('Enter the distance you want a prediction for: ')

closest_time = find_nearest_time(format_time(recorded_time), row_data[distance_run])
closest_column_heading = row_data[distance_run][closest_time]

prediction = [k for k in row_data[predicted_distance].keys()
              if row_data[predicted_distance][k] == closest_column_heading]

print('The predicted time running ' + predicted_distance + ' is: ' + prediction[0] + '.')
```

Import the "format_time()" function from the "tm2secs2tm.py" module.

Use the function to ensure the times used internally by your code are formatted in "HH:MM:SS" format.

Ln: 37 Col: 0



Test Drive

Let's try your code one more time. Hopefully, now that all of the time strings within the system conform to HH:MM:SS format, your code will behave itself.

```

Python Shell
Python 3.1.2 (r312:79360M, Mar 24 2010, 01:33:18)
[GCC 4.0.1 (Apple Inc. build 5493)] on darwin
Type "copyright", "credits" or "license()" for more information.
>>> ===== RESTART =====
>>>
Enter the distance attempted: 20k
Enter the recorded time: 59:59
Enter the distance you want a prediction for: Marathon
Traceback (most recent call last):
  File "/Users/barryp/HeadFirstPython/chapter11/marathon.py", line 31, in <module>
    closest_column_heading = row_data[distance_run][closest_time]
KeyError: '01:00:23'
>>> ===== RESTART =====
>>>
Enter the distance attempted: 20k
Enter the recorded time: 59:59
Enter the distance you want a prediction for: Marathon
The predicted time running Marathon is: 02:14:15.
>>> ===== RESTART =====
>>>
Enter the distance attempted: 5mi
Enter the recorded time: 23:45
Enter the distance you want a prediction for: 10mi
The predicted time running 10mi is: 00:50:02.
>>> ===== RESTART =====
>>>
Enter the distance attempted: 10k
Enter the recorded time: 32:15
Enter the distance you want a prediction for: 25k
The predicted time running 25k is: 01:25:49.
>>>

```

This is the previous test, which crashed with a "KeyError".

This time around, your program behaves itself and works fine.

Another test confirms that things are working well.

And one final test makes sure.

Ln: 31 Col: 4

This is working well. You've solved your application's central problem: your program **reads** in the spreadsheet data from the CSV file, **turns** it into a dictionary of dictionaries, and lets you **interact** with your user to **acquire** the recorded time at a particular distance before **predicting** a time for another distance.

Not counting the code provided by the Head First Code Review Team, you've written fewer than 40 lines of code to solve this problem. *That's quite an achievement.* All that's left to do is to port your program to the club's Android's phones.

And porting to Android won't take too long, will it?

Port to Android

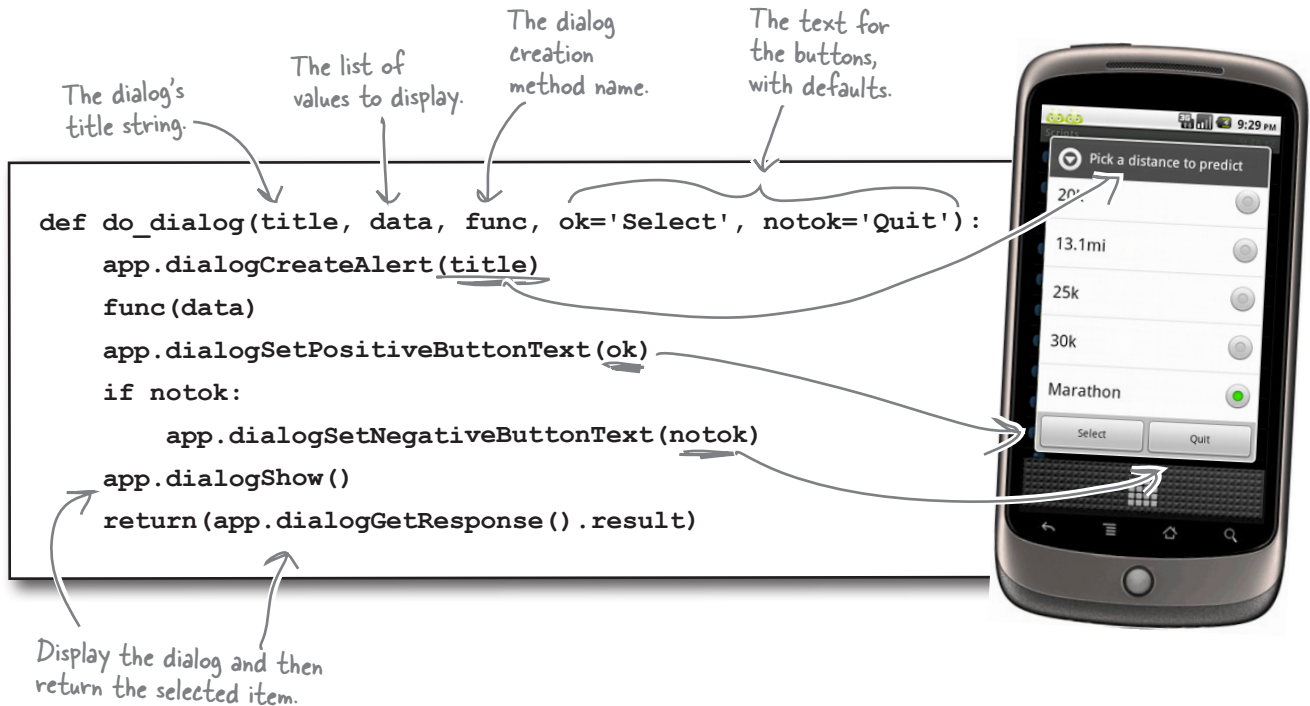
Your code is working great. Now it's time to **port** your text-based Python program to Android. Most of your code doesn't need to change, only the parts that interact with your user.

Obviously, you'll want to make things as easy to use as possible for users of your latest Android app, providing an interface not unlike this one.



Your Android app is a bunch of dialogs

Your Android app interacts with your users through a series of dialogs. Other than the single dialog that requests data *from* your user, the other three share certain similarities. You can take advantage of these shared features by creating a utility function which abstracts the dialog creation details:



Sharpen your pencil

Assume the existence of a list called `distances`, which contains the row distance labels (2mi, 5k, 5mi, and so on). In the space below, provide the two calls to the `do_dialog()` function needed to create the two `dialogSetSingleChoiceItems` shown on the left of the previous page.

.....

.....

.....

.....

.....

.....



Sharpen your pencil Solution

You were to assume the existence of a list called `distances`, which contains the row distance labels. In the space below, you were to provide the two calls to the `do_dialog()` function needed to create the two `dialogSetSingleChoiceItems`.

```
do_dialog("Pick a distance", distances,
         app.dialogSetSingleChoiceItems)
do_dialog("Pick a distance to predict", distances,
         app.dialogSetSingleChoiceItems)
do_dialog('The predicated time running ' + predicted_distance +
         ' is: ', prediction, app.dialogSetItems, "OK", None)
```

Provide the dialog title.

Ditto: do it all again for the other dialog.

Here's another example.

This last one's a little trickier, because you have to build up the dialog title from some variables (that you'll need to have created first).

Use a different dialog creating method this time.

Override the default values for the dialog's buttons.

Provide the list of items to display.

Provide the type of dialog to use.

Get your Android app code ready

To use your dialog creating code, **import** the necessary libraries, **define** some constants, **create** an Android object, and **reuse** some code from earlier in this book:

```
import time
import android
...
distances = [ '2mi', '5k', '5mi', '10k', '15k', '10mi', '20k',
              '13.1mi', '25k', '30k', 'Marathon' ]
...
hello_msg = "Welcome to the Marathon Club's App"
quit_msg = "Quitting the Marathon Club's App."
...
app = android.Android()
def status_update(msg, how_long=2):
    app.makeToast(msg)
    time.sleep(how_long)
```

Do your imports.

Create a list of row labels.

Define two friendly messages.

Create an Android app object.

This function is taken "as-is" from earlier in this book.



Android Pool Puzzle

Your **job** is to take the code from the pool and place it into the blank lines in your Android app code.

You can assume that the `row_data` dictionary exists and has been populated. The variables shown at the bottom of the last page have also been created, and the `status_update()` and `do_dialog()` functions are available to you. Your **goal** is to arrange the code so that it implements the UI interactions you need.

```
status_update(hello_msg)
resp = do_dialog("Pick a distance", distances, ..... )
.....
distance_run = .....
distance_run = distances[distance_run]
..... = app.dialogGetInput("Enter a " + distance_run + " time:",
                          "Use HH:MM:SS format:").result
closest_time = find_nearest_time(format_time( ..... ), row_data[distance_run])
closest_column_heading = row_data[distance_run][closest_time]
resp = do_dialog("Pick a distance to predict", distances, ..... )
.....
predicted_distance = .....
predicted_distance = distances[predicted_distance]
prediction = [k for k in row_data[predicted_distance].keys()
              if row_data[predicted_distance][k] == closest_column_heading]
do_dialog('The predicted time running ' + predicted_distance + ' is: ',
          prediction, app.dialogSetItems, "OK", None)
status_update(quit_msg)
```

The `dialogGetInput()` method displays the input dialog box.



Android Pool Puzzle Solution



Your **job** was to take the code from the pool and place it into the blank lines in your Android app code. You were to assume that the `row_data` dictionary exists and has been populated. The variables you need also have been created, and the `status_update()` and `do_dialog()` functions were available to you. Your **goal** was to arrange the code so that it implements the UI interactions you need.

Ask your user to pick a distance from the list of labels.

```
status_update(hello_msg)
resp = do_dialog("Pick a distance", distances, app.dialogSetSingleChoiceItems)
if resp['which'] in ('positive'):
    distance_run = app.dialogGetSelectedItems().result[0]
    distance_run = distances[distance_run]
    recorded_time = app.dialogGetInput("Enter a " + distance_run + " time:",
                                      "Use HH:MM:SS format:").result
```

Assign the selected distance label to "distance_run".

Ask your user enter the recorded time.

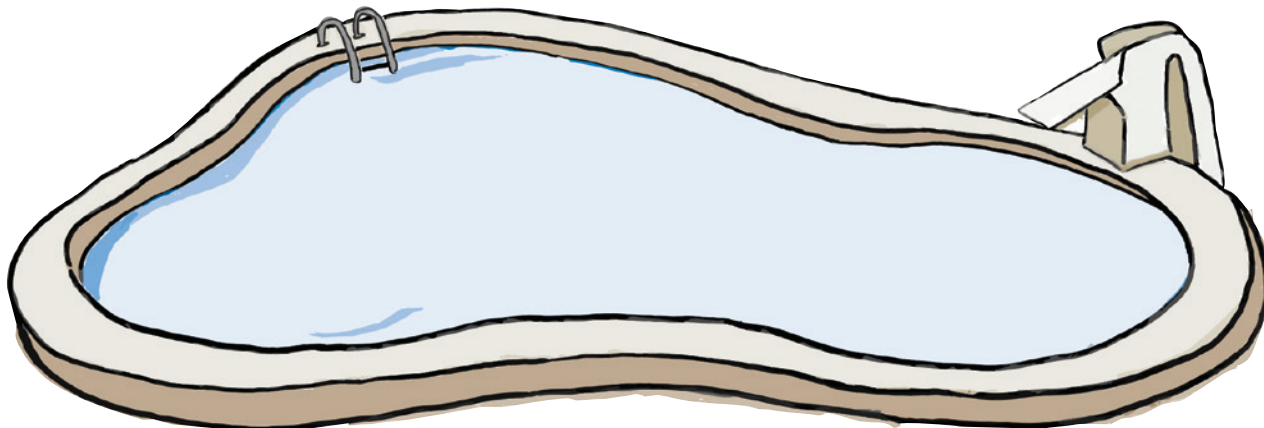
Work out what column heading to use.

Look up the prediction.

```
closest_time = find_nearest_time(format_time(recorded_time), row_data[distance_run])
closest_column_heading = row_data[distance_run][closest_time]
resp = do_dialog("Pick a distance to predict", distances, app.dialogSetSingleChoiceItems)
if resp['which'] in ('positive'):
    predicted_distance = app.dialogGetSelectedItems().result[0]
    predicted_distance = distances[predicted_distance]
    prediction = [k for k in row_data[predicted_distance].keys()
                  if row_data[predicted_distance][k] == closest_column_heading]
    do_dialog('The predicted time running ' + predicted_distance + ' is: ',
              prediction, app.dialogSetItems, "OK", None)
status_update(quit_msg)
```

Ask your user to pick a distance from the list of labels to predict to.

Display the predicted time at the selected distance to your user.



Put your app together...

You now have all the code you need to create your app:

```

marathonapp.py - /Users/barryp/HeadFirstPython/chapter11/marathonapp.py

import time
import android

from find_it import find_closest
from tm2secs2tm import time2secs, secs2time, format_time

def find_nearest_time(look_for, target_data):
    what = time2secs(look_for)
    where = [time2secs(t) for t in target_data]
    res = find_closest(what, where)
    return(secs2time(res))

distances = [ '2mi', '5k', '5mi', '10k', '15k', '10mi', '20k',
              '13.1mi', '25k', '30k', 'Marathon' ]
hello_msg = "Welcome to the Marathon Club's App"
quit_msg = "Quitting the Marathon Club's App."

row_data = {}

with open('/sdcard/s14a/scripts/PaceData.csv') as paces:
    column_headings = paces.readline().strip().split(',')
    column_headings.pop(0)
    for each_line in paces:
        row = each_line.strip().split(',')
        row_label = row.pop(0)
        inner_dict = {}
        for i in range(len(column_headings)):
            inner_dict[format_time(row[i])] = column_headings[i]
        row_data[row_label] = inner_dict

app = android.Android()

def status_update(msg, how_long=2):
    app.makeToast(msg)
    time.sleep(how_long)

def do_dialog(title, data, func, ok='Select', notok='Quit'):
    app.dialogCreateAlert(title)
    func(data)
    app.dialogSetPositiveButton(ok)
    if notok:
        app.dialogSetNegativeButton(notok)
    app.dialogShow()
    return(app.dialogGetResponse().result)

status_update(hello_msg)

resp = do_dialog("Pick a distance", distances, app.dialogSetSingleChoiceItems)
if resp['which'] in ('positive'):
    distance_run = app.dialogGetSelectedItem().result[0]
    distance_run = distances[distance_run]
    recorded_time = app.dialogGetInput("Enter a " + distance_run + " time:",
                                      "Use HH:MM:SS format:").result
    closest_time = find_nearest_time(format_time(recorded_time), row_data[distance_run])
    closest_column_heading = row_data[distance_run][closest_time]
    resp = do_dialog("Pick a distance to predict", distances, app.dialogSetSingleChoiceItems)
    if resp['which'] in ('positive'):
        predicted_distance = app.dialogGetSelectedItem().result[0]
        predicted_distance = distances[predicted_distance]
        prediction = [k for k in row_data[predicted_distance].keys()
                      if row_data[predicted_distance][k] == closest_column_heading]
        do_dialog('The predicted time running ' + predicted_distance + ' is: ' +
                  prediction, app.dialogSetItems, "OK", None)

status_update(quit_msg)

```

Do your imports.

Include your "find_nearest()" function.

Declare your constants.

NOTE: the location of the data file on the SDCARD is specific to Android.

Grab and preprocess your CSV data.

Create your Android app object and include your helper functions.

Display your UI to your user and process the resulting interaction.

Ln: 67 Col: 0



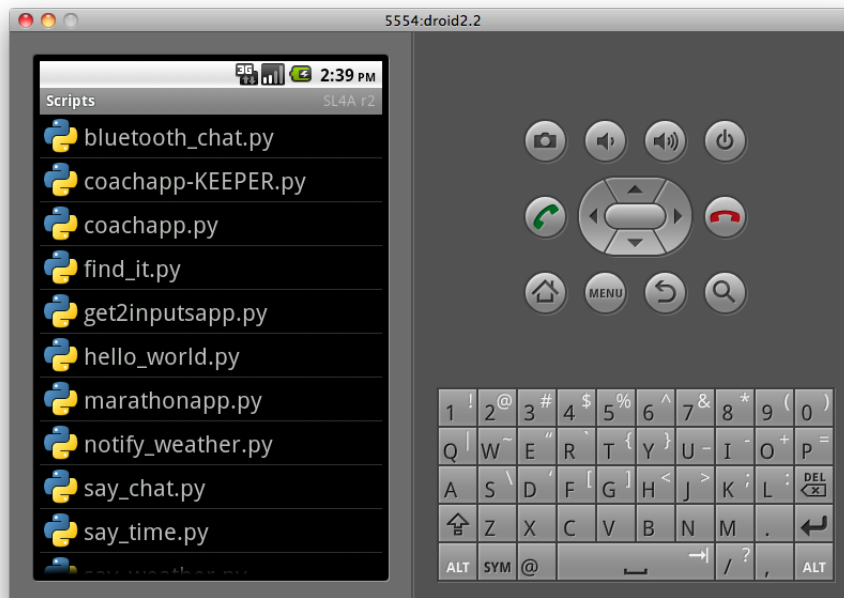
Test Drive

It's time to test your Android app on the Android Emulator before loading a working application onto a "real" phone. Start your Android emulator and begin by transferring your code and the files it needs onto the emulator's *SDCARD*. Use the `adb` command in the `tools` folder to copy `marathonapp.py`, `find_it.py`, `tm2sec2tm.py` and `PaceData.csv` to the emulator, and then take your app for a spin.

Copy your code and its support files to the emulator with these commands.

```
File Edit Window Help CopyToEmulator
$ tools/adb push marathonapp.py /mnt/sdcard/sl4a/scripts
43 KB/s (2525 bytes in 0.056s)
$ tools/adb push find_it.py /mnt/sdcard/sl4a/scripts
7 KB/s (555 bytes in 0.069s)
$ tools/adb push tm2secs2tm.py /mnt/sdcard/sl4a/scripts
12 KB/s (628 bytes in 0.050s)
$ tools/adb push PaceData.csv /mnt/sdcard/sl4a/scripts
59 KB/s (4250 bytes in 0.069s)
```

And there it is... waiting for you to test it.



Go on. You know you want to: tap that app!

Your app's a wrap!

All that's left to do is transfer your working Android app to the Marathon Club's phones...and that's easy when you use **AndFTP**. When you show off your latest work, the club's members can't believe their eyes.



And there's no stopping you!

You've put your Python skills and techniques to great use here.

Whether you're building an app for the smallest handheld device or the biggest web server, your Python skills help you get the job done.

Congratulations!



Your Python Toolbox

You've got Chapter 11 under your belt and you've demonstrated a mastery of your Python toolbox. Congratulations and well done!

Python Lingo

- A “conditional” list comprehension is one that includes a trailing “if” statement, allowing you to control which items are added to the new list as the comprehension runs.
- List comprehensions can be rewritten as an equivalent “for” loop.



BULLET POINTS

- The `input()` BIF lets you prompt and receive input from your users.
- If you find yourself using **Python 2** and in need of the `input()` function, use the `raw_input()` function instead.
- Build complex data structures by combining Python's built-in lists, sets, and dictionaries.
- The `time` module, which is part of the standard library, has a number of functions that make converting between time formats possible.

It's time to go...

It's been a blast having you with us here on Lake Python. Come back soon and often. We love it when you drop by.



This is just the beginning

We're sad to see you leave, but there's nothing like taking what you've learned and putting it to use. You're just beginning your Python journey and you're in the driver's seat. We're dying to hear how things go, so *drop us a line* at the Head First Labs website, www.headfirstlabs.com, and let us know how Python is paying off for **YOU!**

